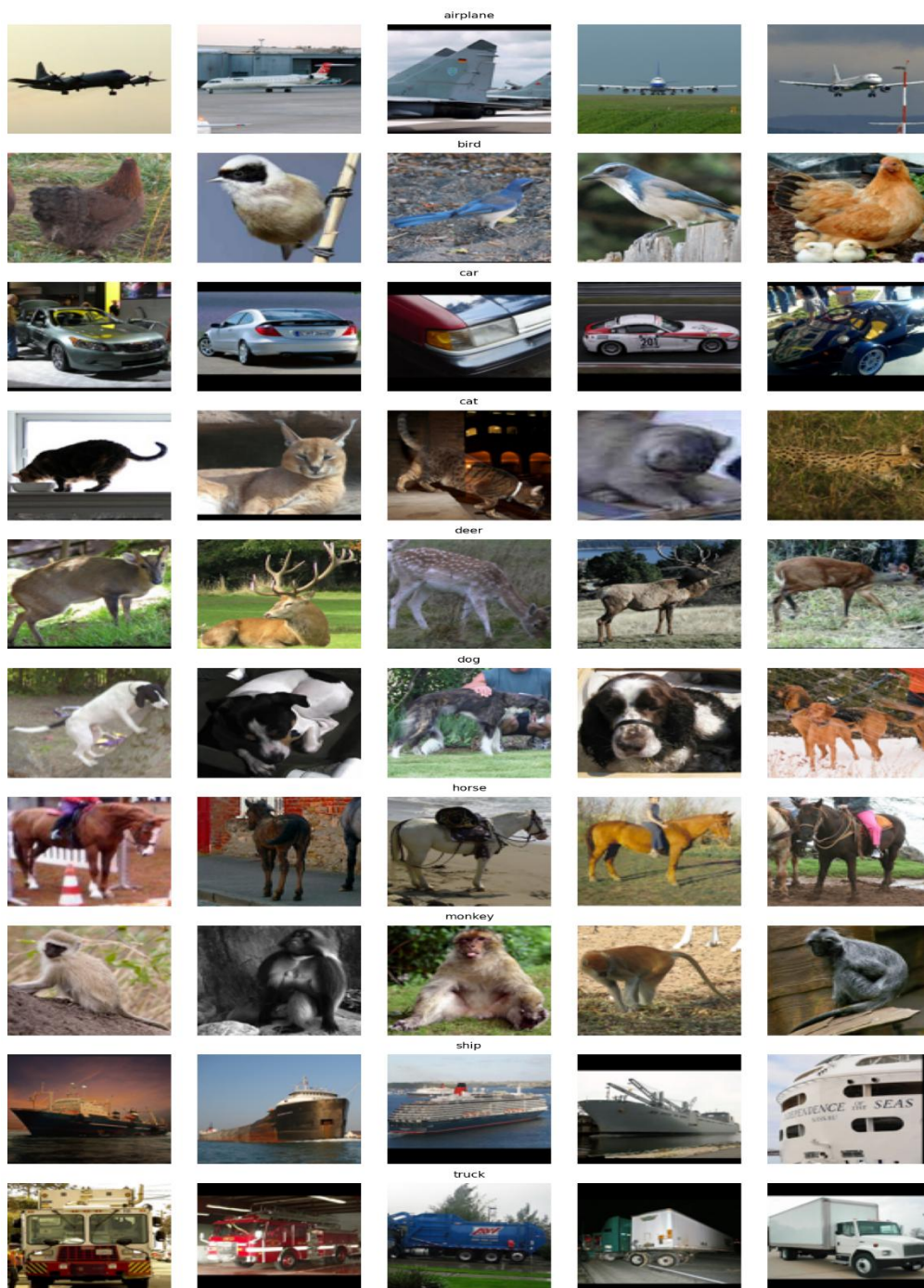


Image Classification Analysis

Vlad Schiopu

1.1 Data exploration

Imagebits



The classes in the datasets are balanced and there are the same number of examples for each class: 800 for each class in the train and 500 for each class in the test

Intra-class similarity

The examples are very varied. The images capture the object in different positions, from different distances, with different backgrounds and with different lighting. The classification model must be robust and take into account all these differences.

Similaritate Inter-class

We can see some examples that can be very similar in some classes:

- Cat, Dog, Deer and Horse are generally caught standing on the ground and are surrounded by nature.

- Monkey and bird can look like each other because they sit in trees

- airplane and ship are first resembled by the presence of the sky and there are some examples of airplanes in which water also appears

- car and truck are found in many examples on the road and there may be car/truck models that are similar and unclear enough to be confused

In addition, the low resolution of the photos makes it difficult to detect details.

Land Patches

AnnualCrop



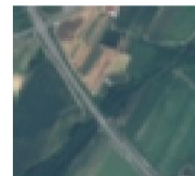
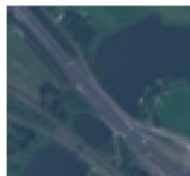
Forest



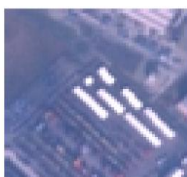
HerbaceousVegetation



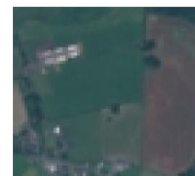
Highway



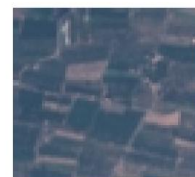
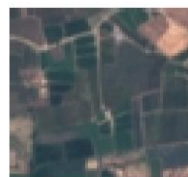
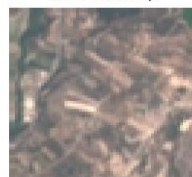
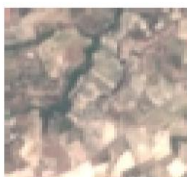
Industrial



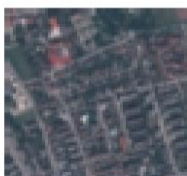
Pasture



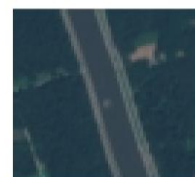
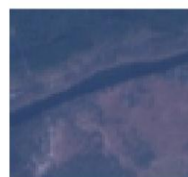
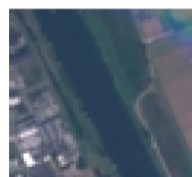
PermanentCrop



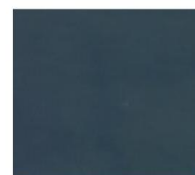
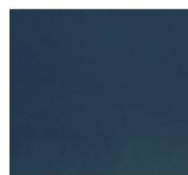
Residential



River



SeaLake



The classes in the datasets are balanced, and there are the same number of examples for each class. In the dataset land_patches there are 200 examples per class in the train set, and in the test set there are 700. This is a bit suspicious because I would have expected more data on the train.

Intra-class similarity

Here the examples in each class do not vary so much between them. There are some clear patterns in some classes such as:

- Forest (nuanta uniforma de verde)
- SeaLake (nuanta uniforma de albastru)
- Highway (curbe gri)

Similaritate Inter-class

There are some slight similarities between classes such as:

- Forest si Pasture (predominant verde)
- Forest and SeaLake (lack of details)
- AnnualCrop and PermanentCrop (organization in rectangles and similar colors)
- Forest and SeaLake (lack of details)
- Highway si river (forma similara de linii)

1.2 Neural Network Training

We used resize, crop, colorjitter, and horisontalFlip augmentations to help the model generalize better, especially on a dataset with very high in-class variation. For LandPatches we gave up ColorJitter because color is important for model prediction and there is little data.

MLP_Imagebits

Main architecture:

```
class MLP_ImageBits(nn. Module):
    def __init__(self, num_classes=10):
        super().__init__()

        self.flatten = nn. Flatten()

        self.fc1 = nn. Linear(3 * 96 * 96, 1024)
        self.fc2 = nn. Linear(1024, 512)
        self.fc3 = nn. Linear(512, num_classes)

        self.relu = nn. ReLU()
        self.dropout = nn. Dropout(p=0.3)

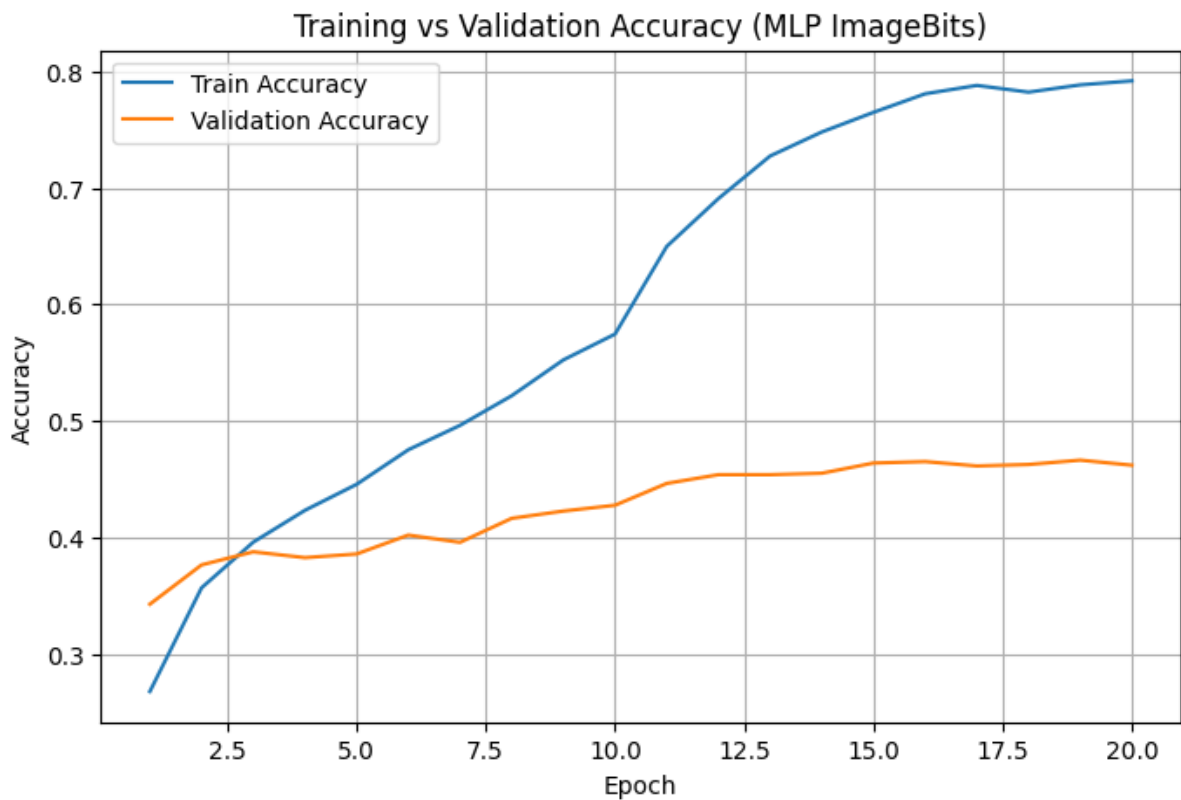
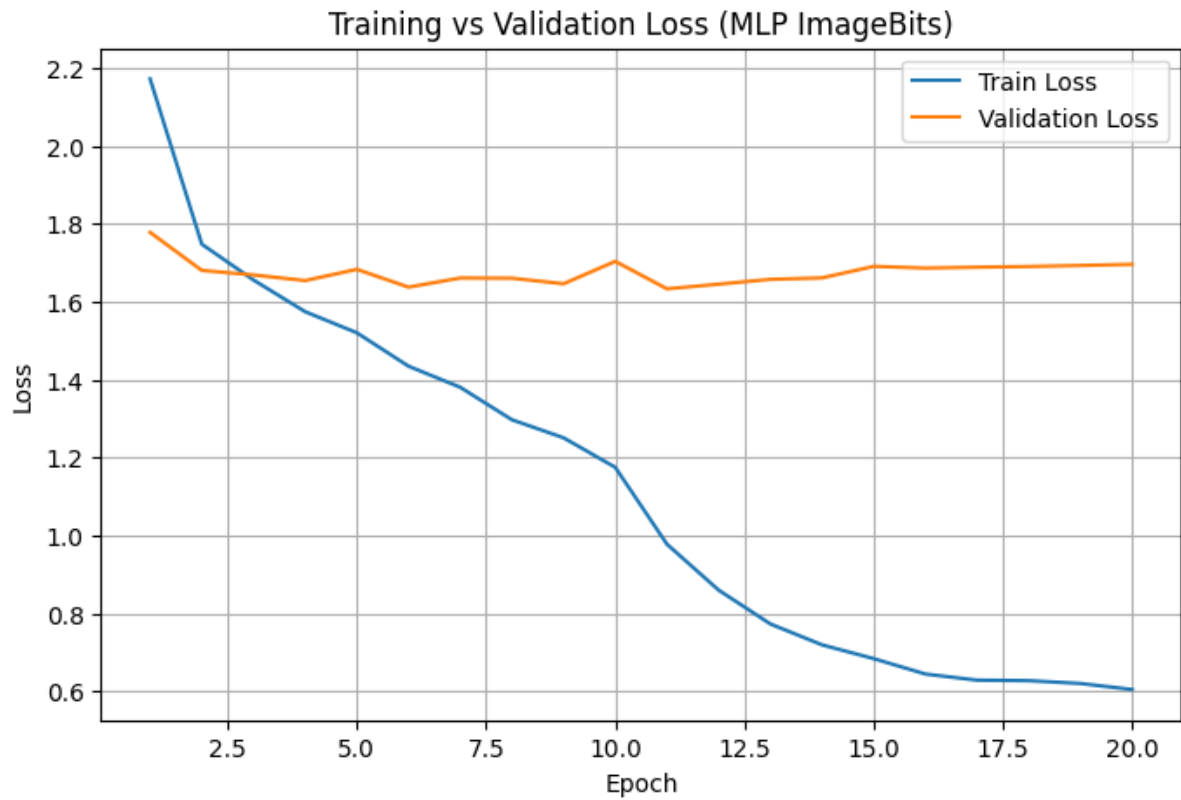
        def forward(self, x):
            x = self.flatten(x)
            x = self.relu(self.fc1(x))
            x = self.dropout(x)
            x = self.relu(self.fc2(x))
            x = self.dropout(x)
            x = self.fc3(x)
            return x
```

-Simple:

Epoch 20 | Train loss: 0.6045, acc: 0.7920 | Val loss: 1.6958, acc: 0.4619

Test Accuracy: 0.4410

Test F1-score (macro): 0.4407

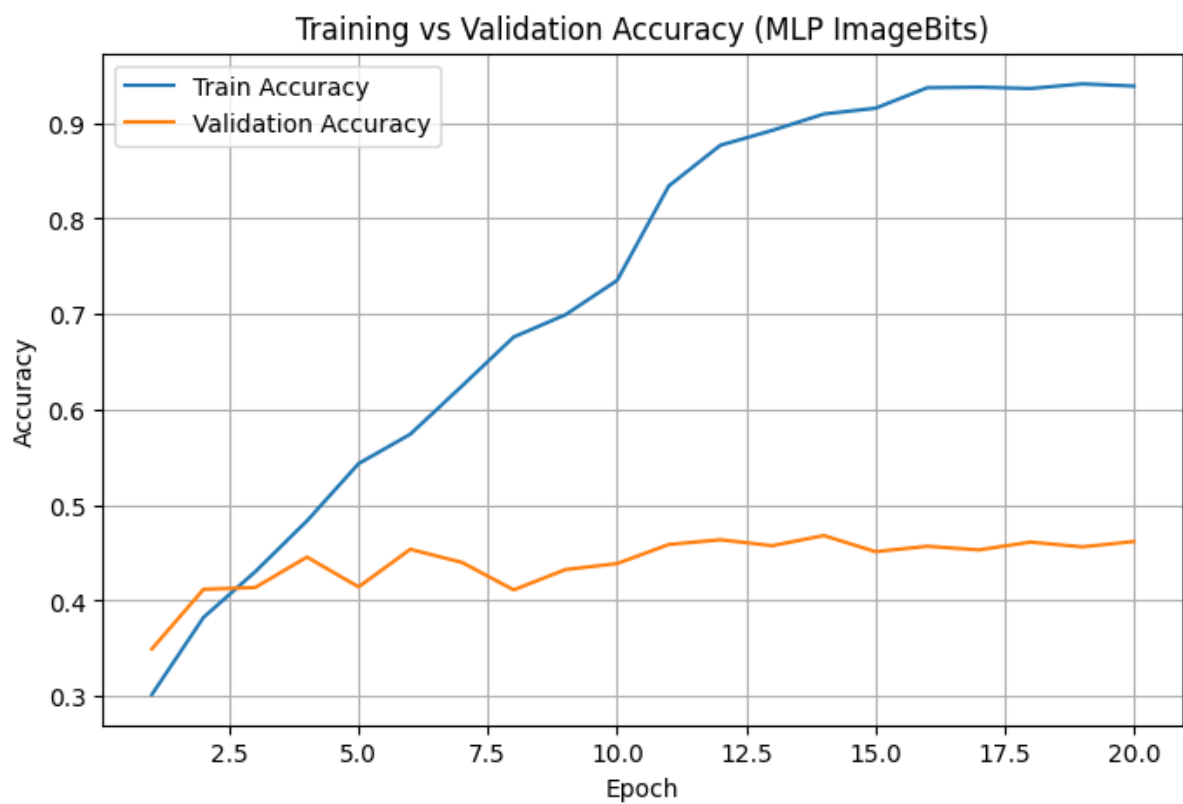
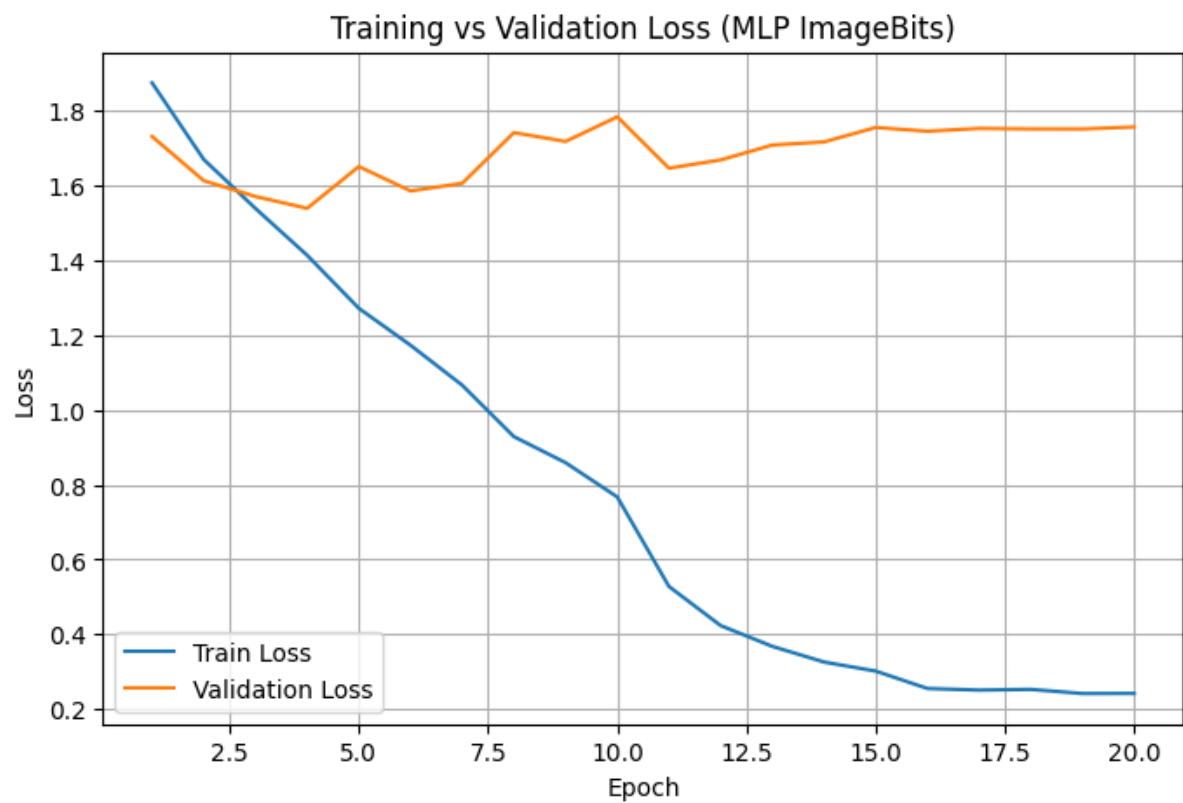


-With BatchNorm:

Epoch 20 | Train loss: 0.2419, acc: 0.9386 | Val loss: 1.7566, acc: 0.4619

Test Accuracy: 0.4622

Test F1-score (macro): 0.4634

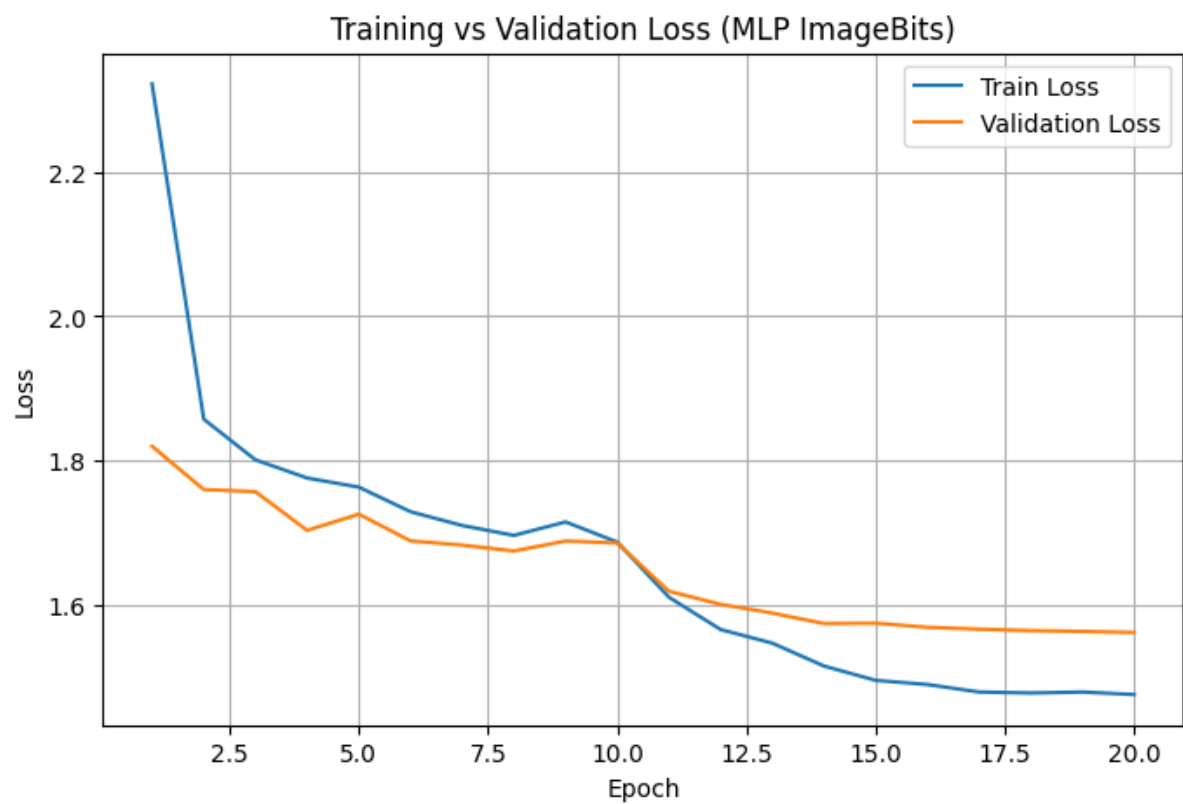


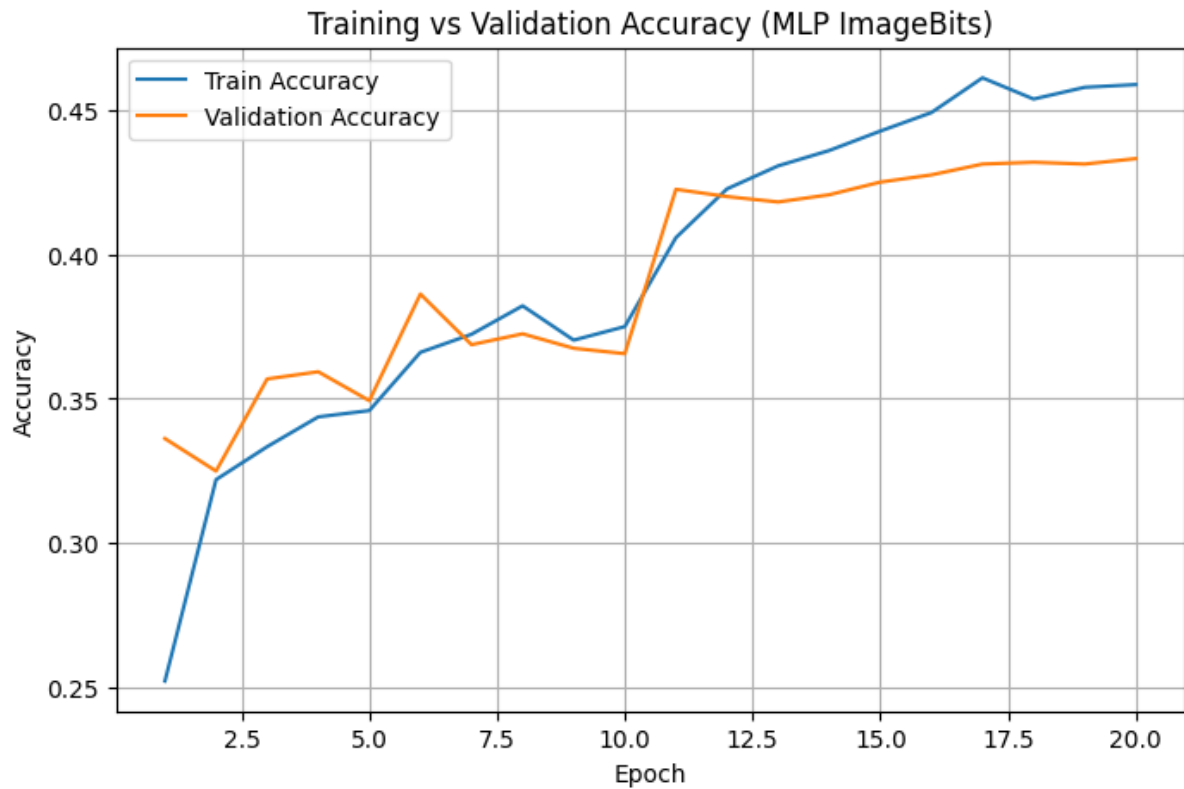
-With Augumentations:

Epoch 20 | Train loss: 1.4757, acc: 0.4587 | Val loss: 1.5618, acc: 0.4331

Test Accuracy: 0.4392

Test F1-score (macro): 0.4290





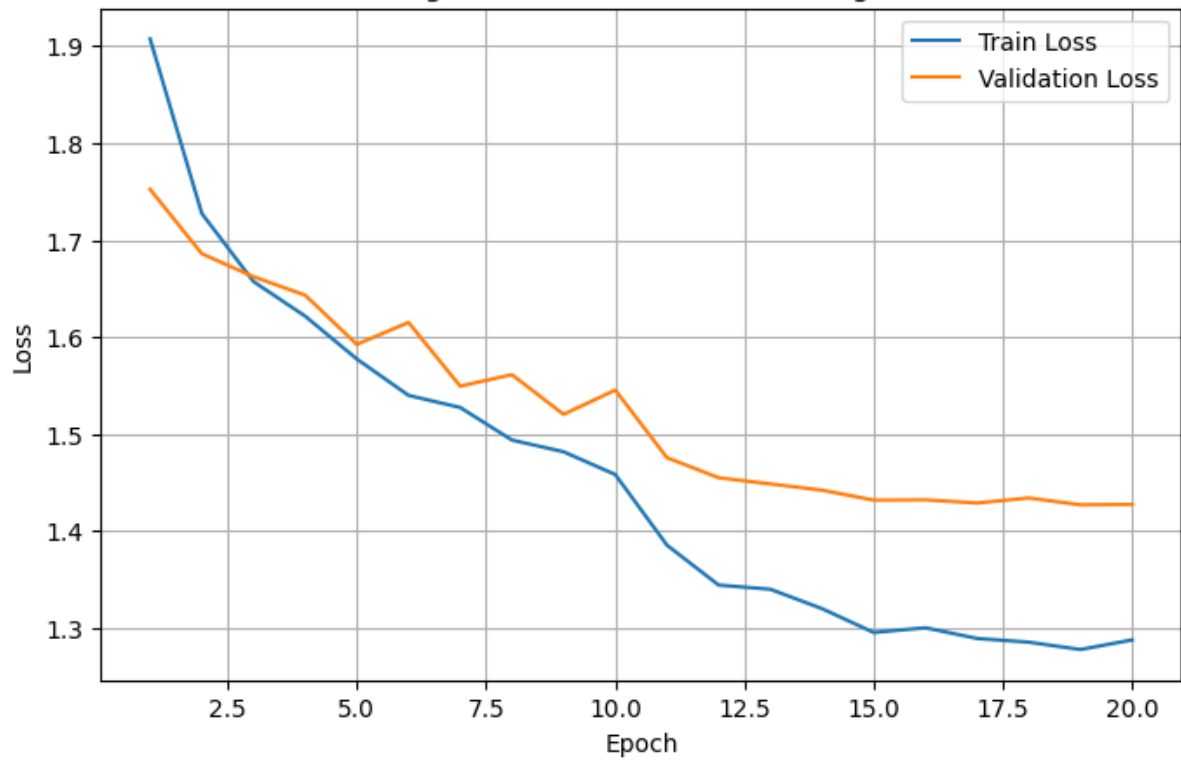
-With Both:

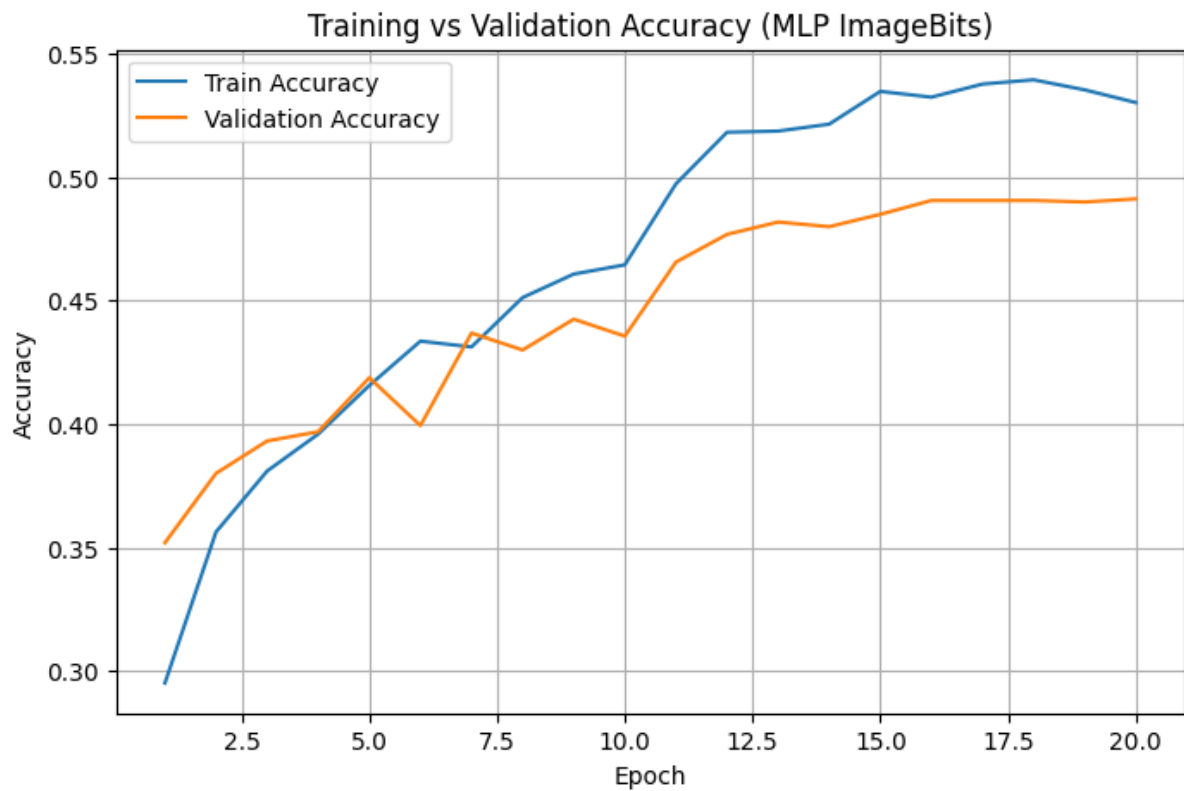
Epoch 20 | Train loss: 1.2879, acc: 0.5303 | Val loss: 1.4275, acc: 0.4913

Test Accuracy: 0.4894

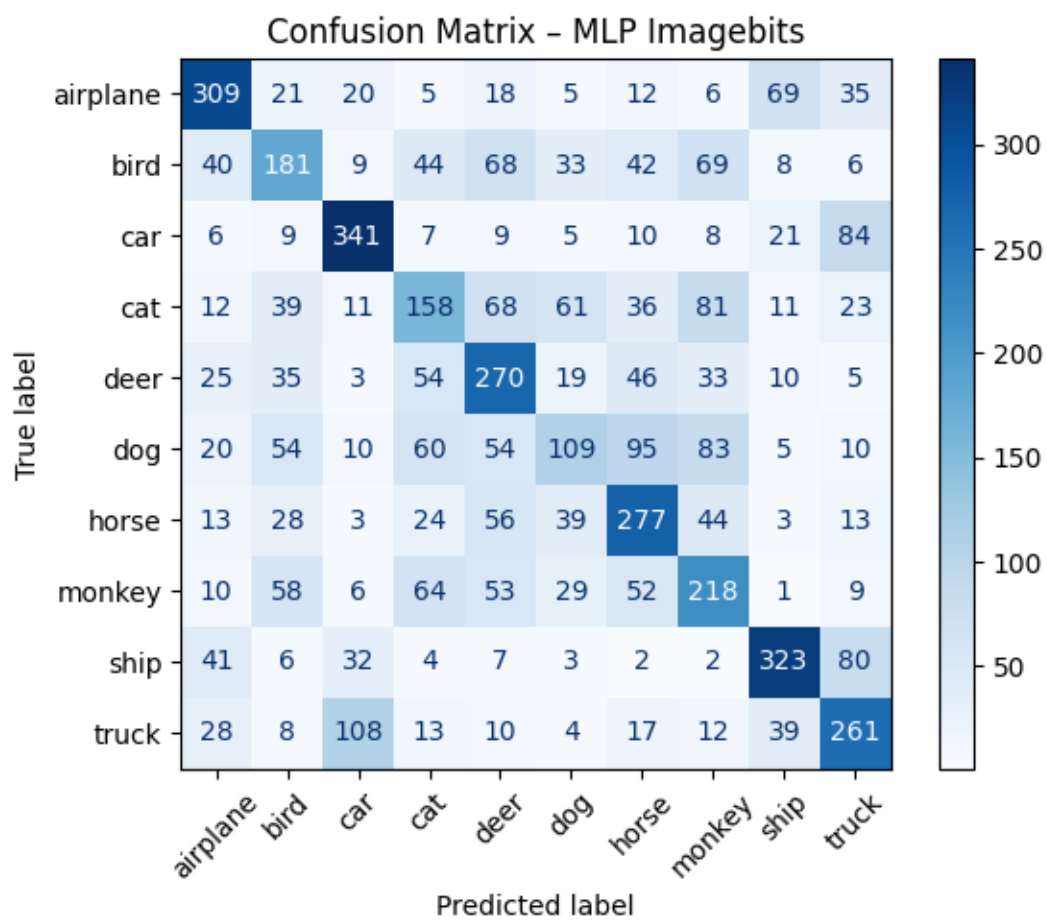
Test F1-score (macro): 0.4832

Training vs Validation Loss (MLP ImageBits)





Best: Both



CNN_Imagebits

Main architecture:

```
class CNN_ImageBits(nn. Module):
    def __init__(self, num_classes=10):
        super().__init__()

        self.conv_block1 = nn. Sequential(
            nn. Conv2d(3, 32, kernel_size=3, padding=1),
            nn. ReLU(),
            nn. MaxPool2d(2)
        )

        self.conv_block2 = nn. Sequential(
            nn. Conv2d(32, 64, kernel_size=3, padding=1),
            nn. ReLU(),
            nn. MaxPool2d(2)
        )

        self.conv_block3 = nn. Sequential(
            nn. Conv2d(64, 128, kernel_size=3, padding=1),
            nn. ReLU(),
            nn. MaxPool2d(2)
        )

        self.flatten = nn. Flatten()

        self.fc1 = nn. Linear(128 * 12 * 12, 512)
        self.relu = nn. ReLU()
        self.dropout = nn. Dropout(p=0.5)

        self.fc2 = nn. Linear(512, num_classes)

    def forward(self, x):
        x = self.conv_block1(x)
```

```
x = self.conv_block2(x)
x = self.conv_block3(x)
```

```
x = self.flatten(x)
```

```
x = self.fc1(x)
x = self.relu(x)
x = self.dropout(x)
```

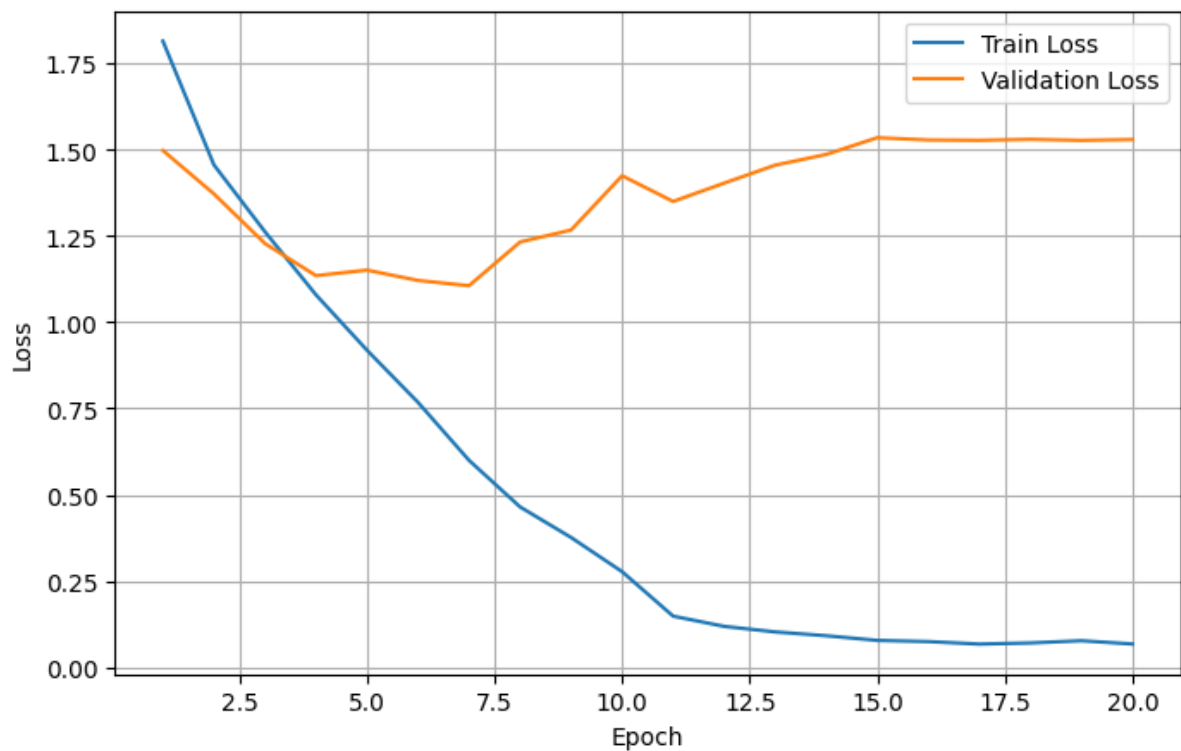
```
x = self.fc2(x)
return x
```

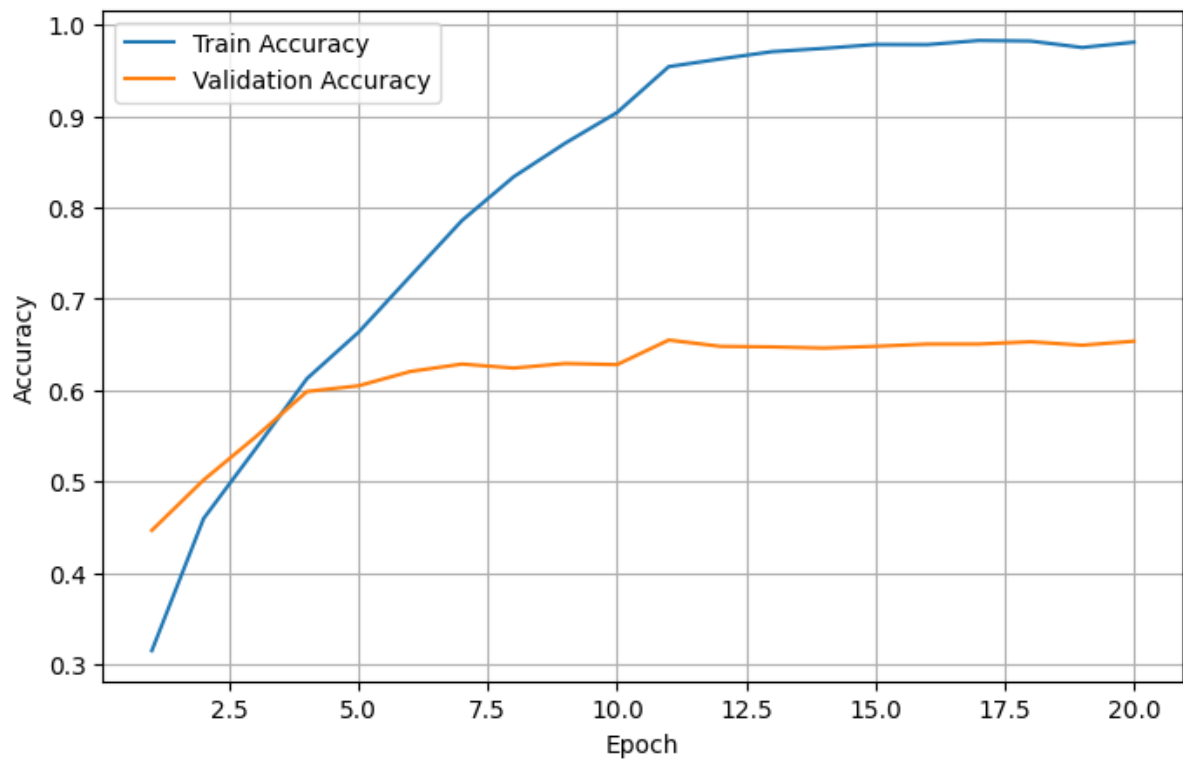
-Simple:

Epoch 20 | Train loss: 0.0668, acc: 0.9808 | Val loss: 1.5302, acc: 0.6538

Test Accuracy: 0.6648

Test F1-score (macro): 0.6648



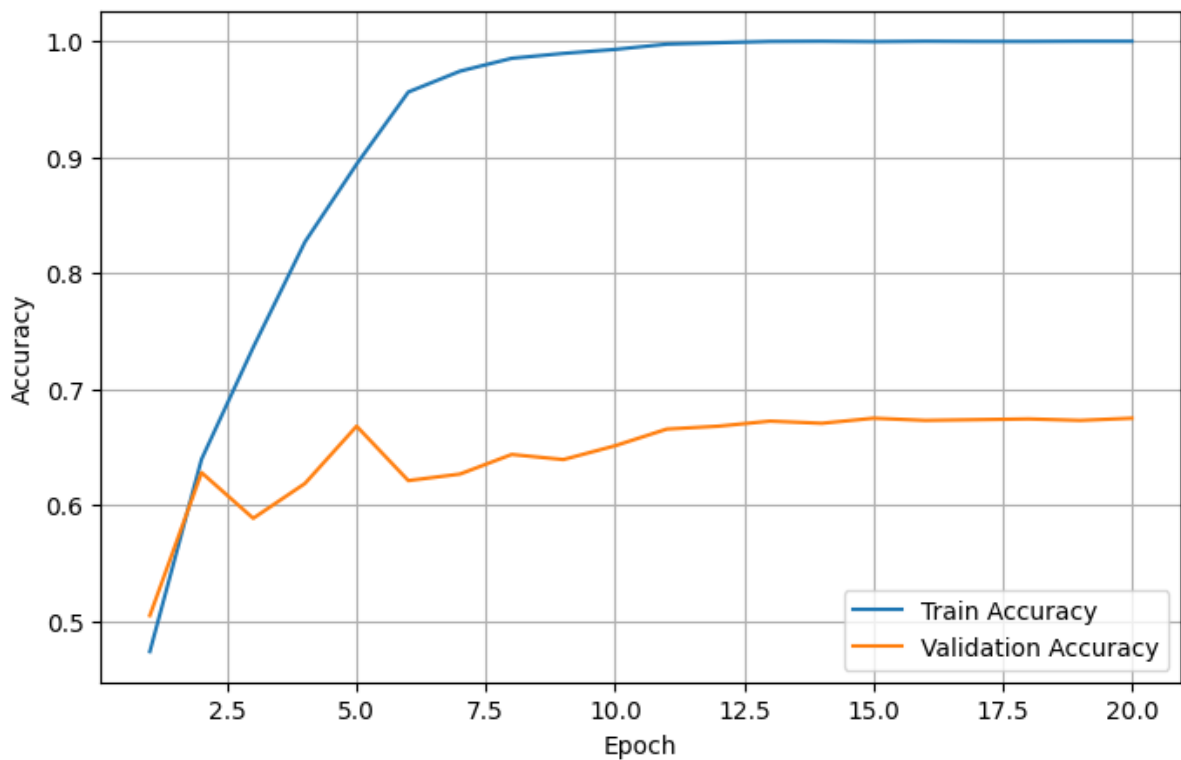
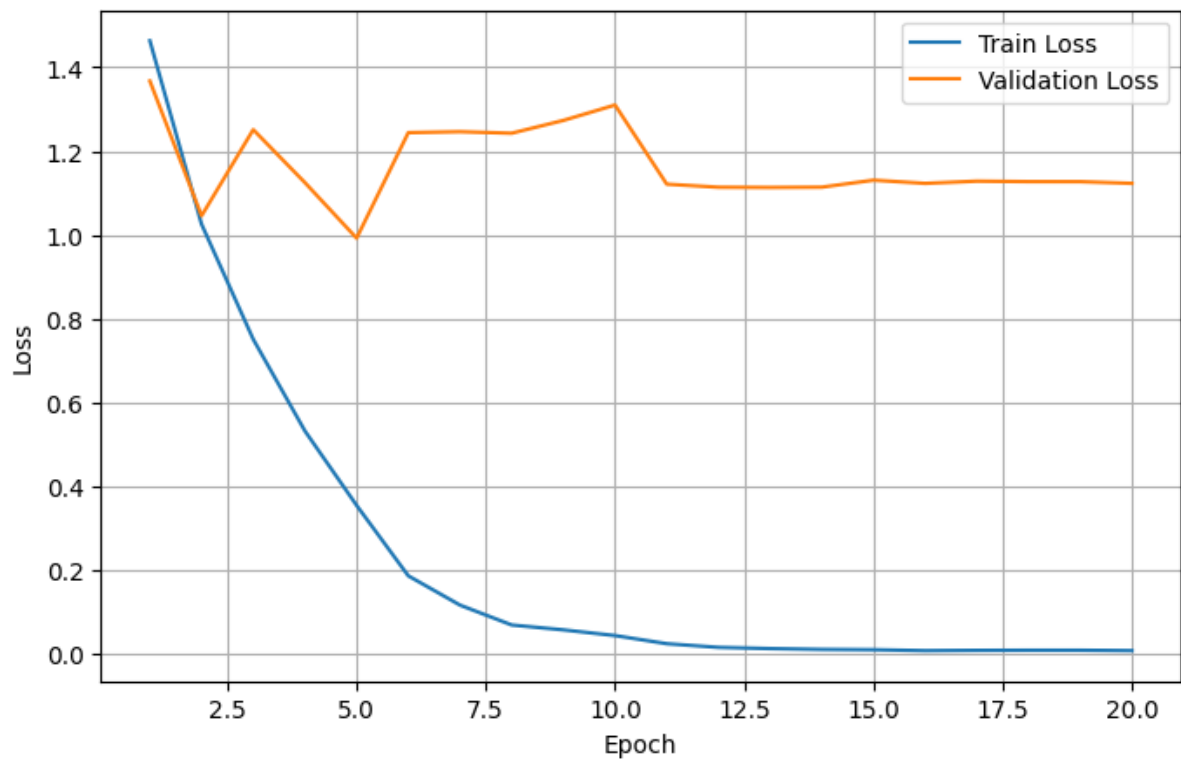


-With BatchNorm:

Epoch 20 | Train loss: 0.0089, acc: 0.9998 | Val loss: 1.1233, acc: 0.6750

Test Accuracy: 0.6800

Test F1-score (macro): 0.6792

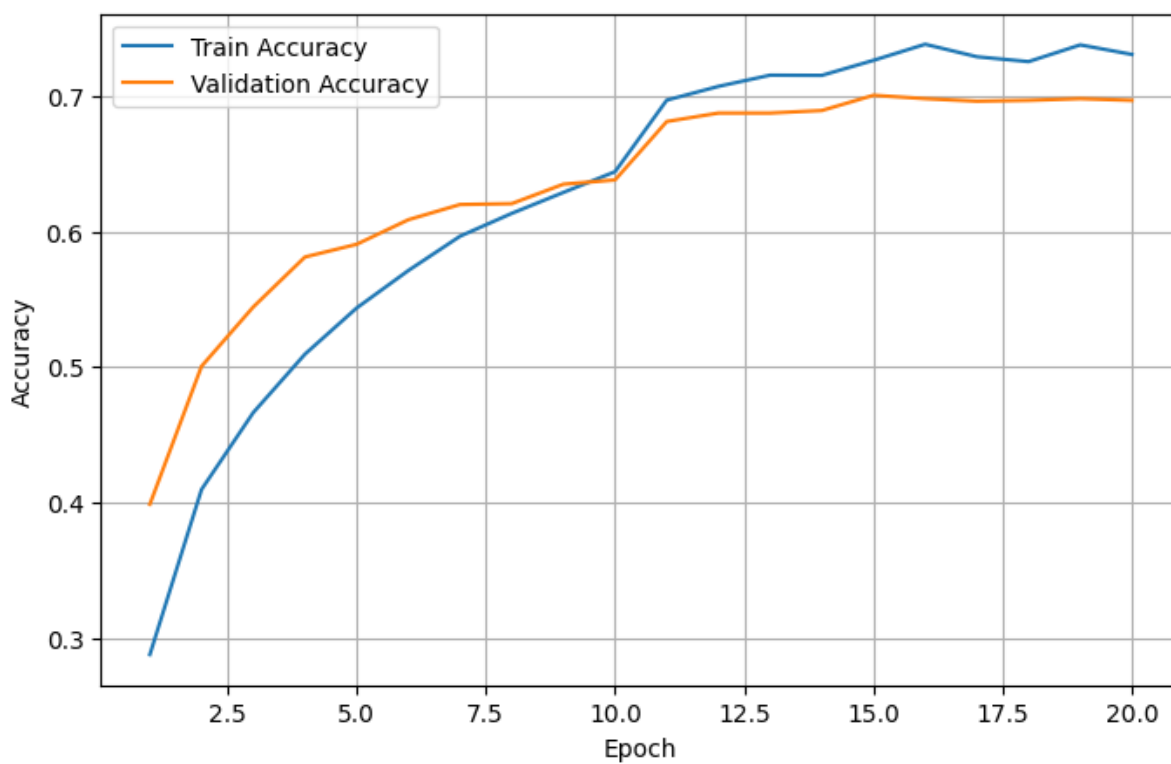
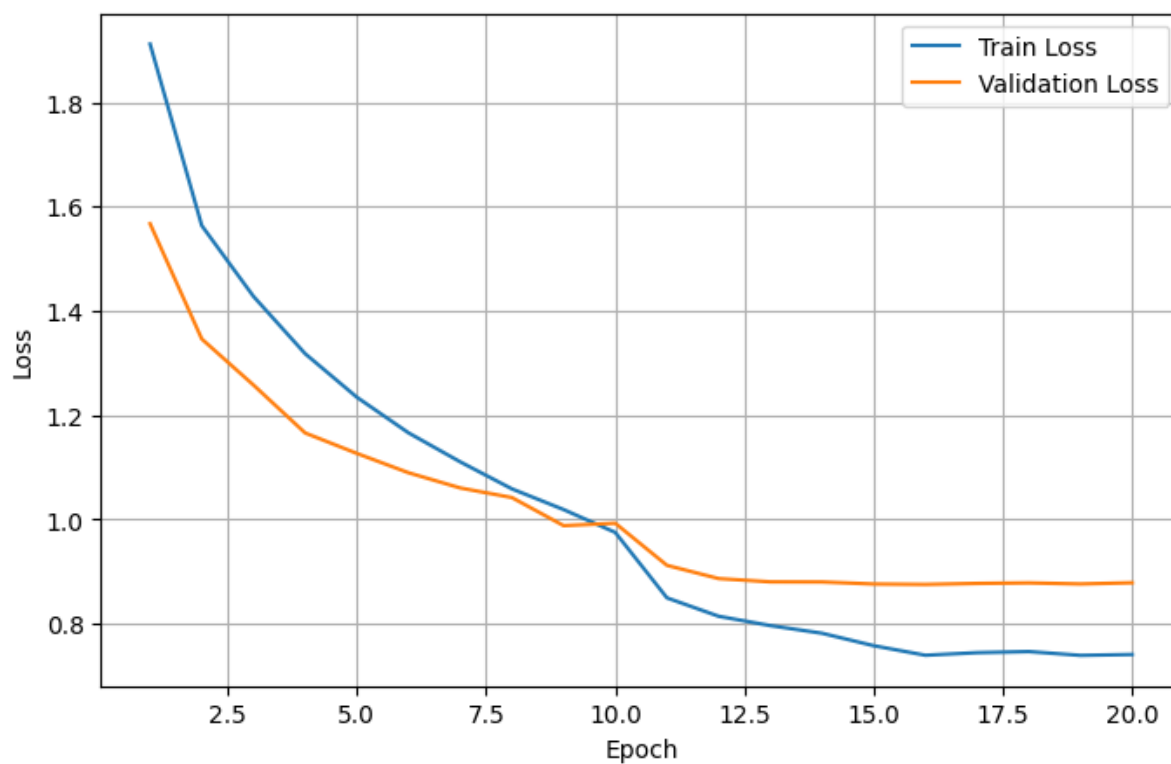


-With Augmentations:

Epoch 20 | Train loss: 0.7410, acc: 0.7308 | Val loss: 0.8786, acc: 0.6969

Test Accuracy: 0.6802

Test F1-score (macro): 0.6762

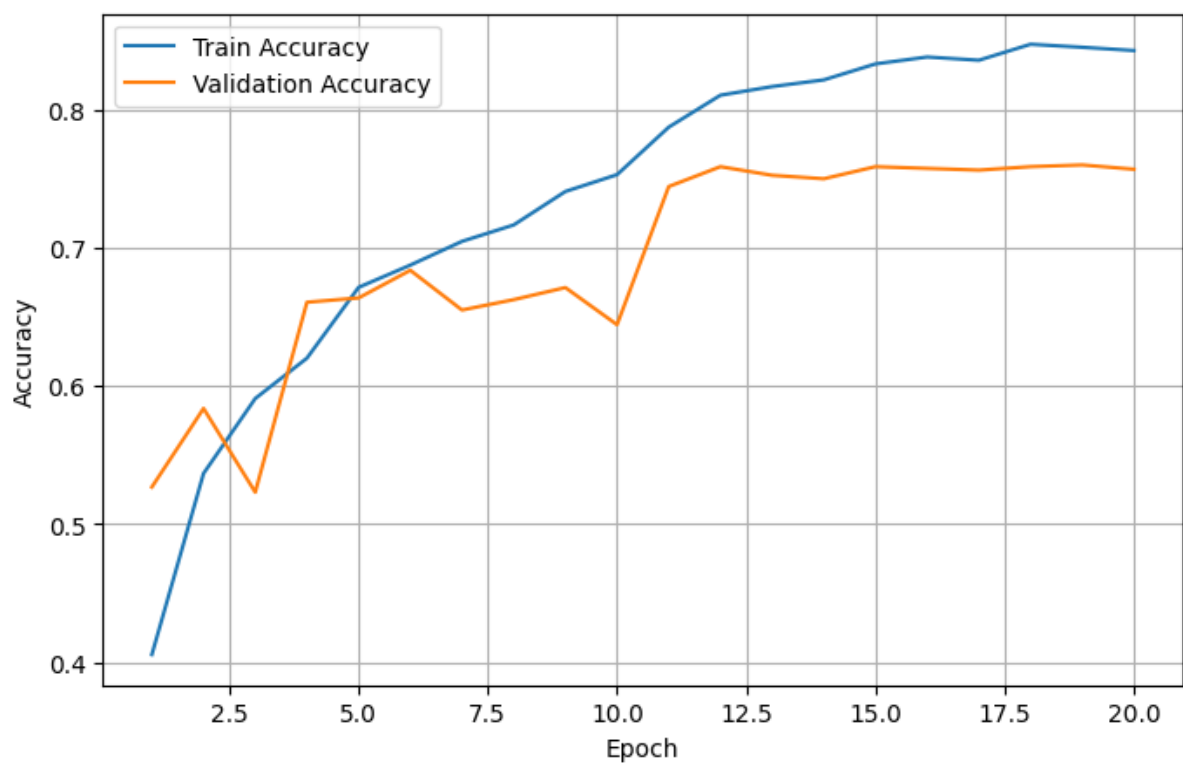
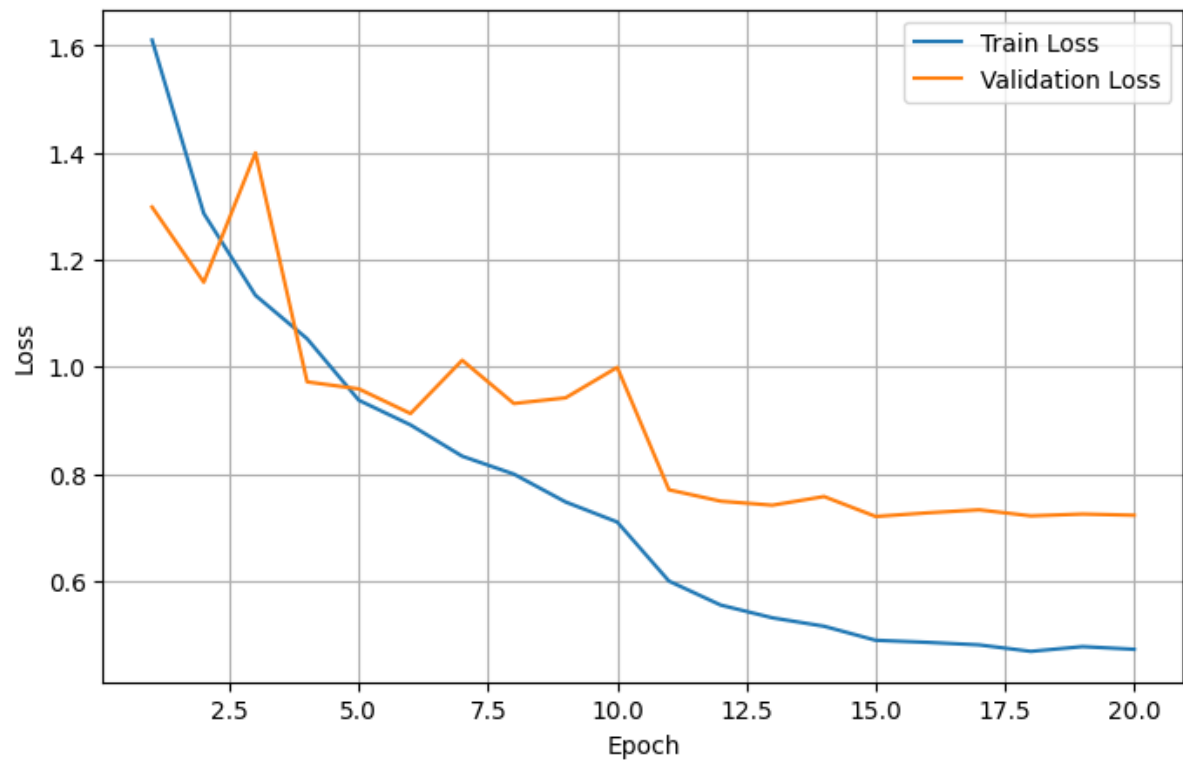


-With Both:

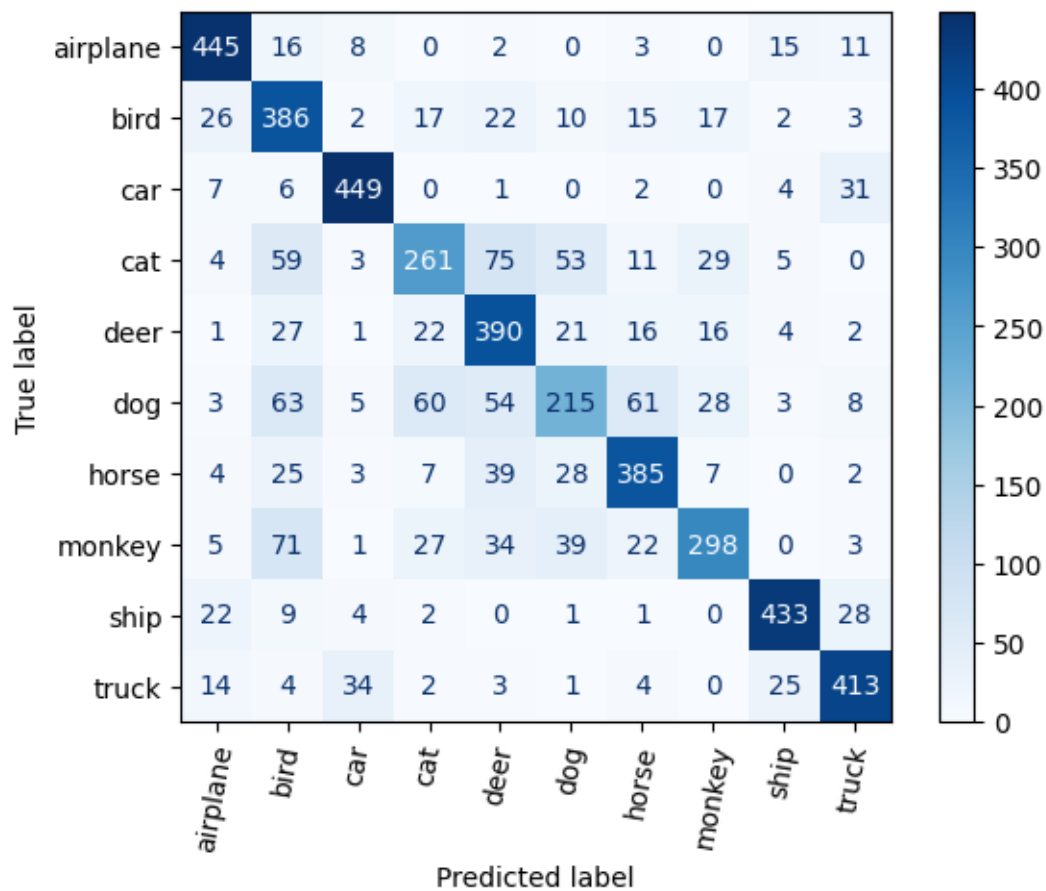
Epoch 20 | Train loss: 0.4727, acc: 0.8428 | Val loss: 0.7231, acc: 0.7569

Test Accuracy: 0.7350

Test F1-score (macro): 0.7307



Best: Both



Bottom line: Augmentations are crucial and drastically reduce overfitting.

Table Imagebits:

Configuration	Acuratete maxima train	Acuratete test
MLP Simple	0.7920	0.4410
MLP With augmentations	0.4587	0.4392
MLP With Batchnorm	0.9386	0.4634
MLP With both	0.5303	0.4894
CNN Simple	0.9808	0.6648
CNN With augmentations	0.7308	0.6802
CNN With Batchnorm	0.9998	0.6800
CNN With both	0.8428	0.7350

Configurations without boosts have resulted in overfitting, which is even stronger when batchnorm is used. However, batchnorm works well with augmentations and leads to good balance and accuracy.

LandPatches

To use the transfer learning procedure I resized all the images in the LandPatches dataset to 96x96 and I was able to use the architectures from imagebits but I removed weights and bias from the classification layers.

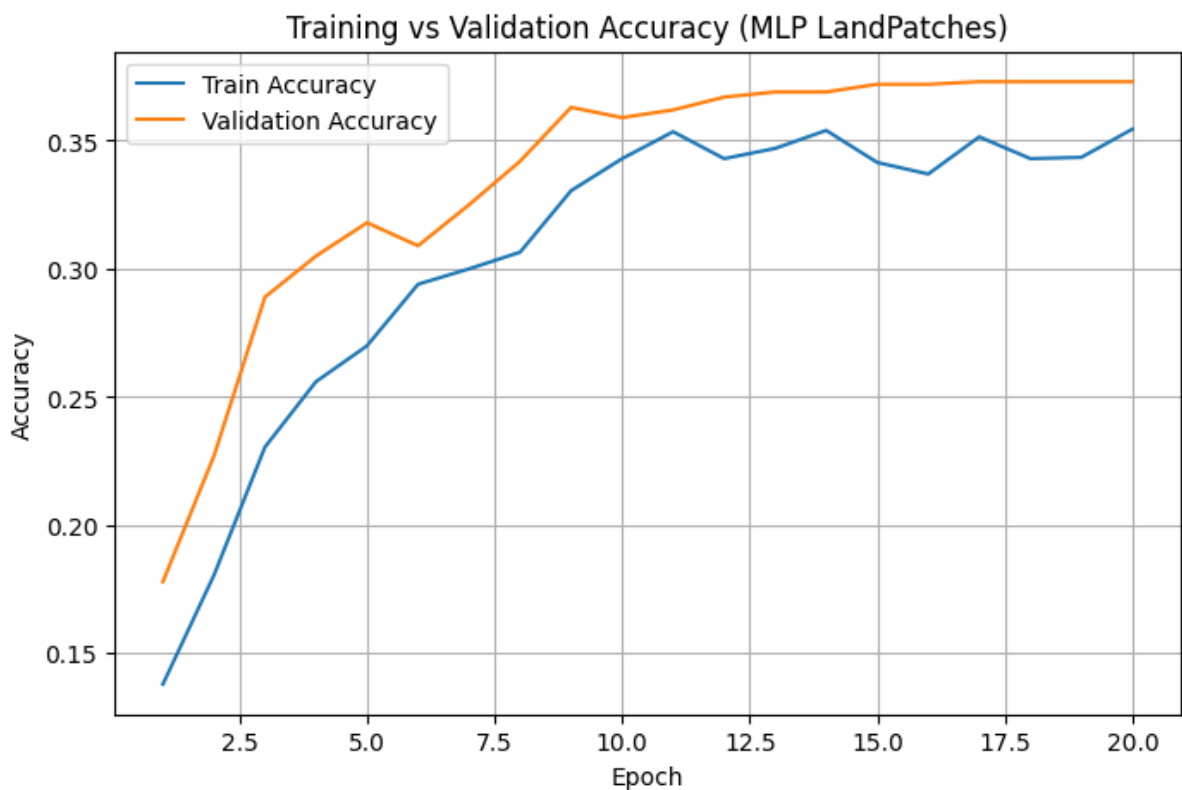
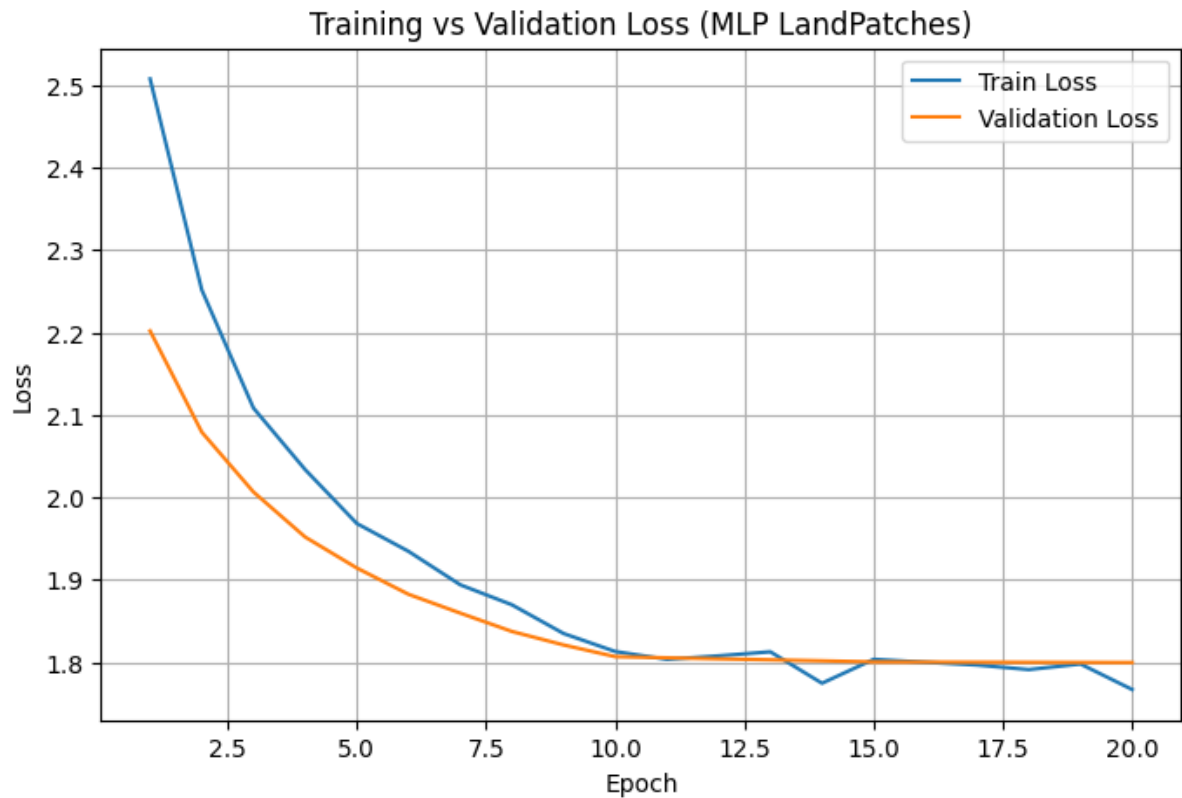
MLP_LandPatches

-Simple:

Epoch 20 | Train loss: 1.7668, acc: 0.3545 | Val loss: 1.7996, acc: 0.3730

Test Accuracy: 0.3834

Test F1-score (macro): 0.3531

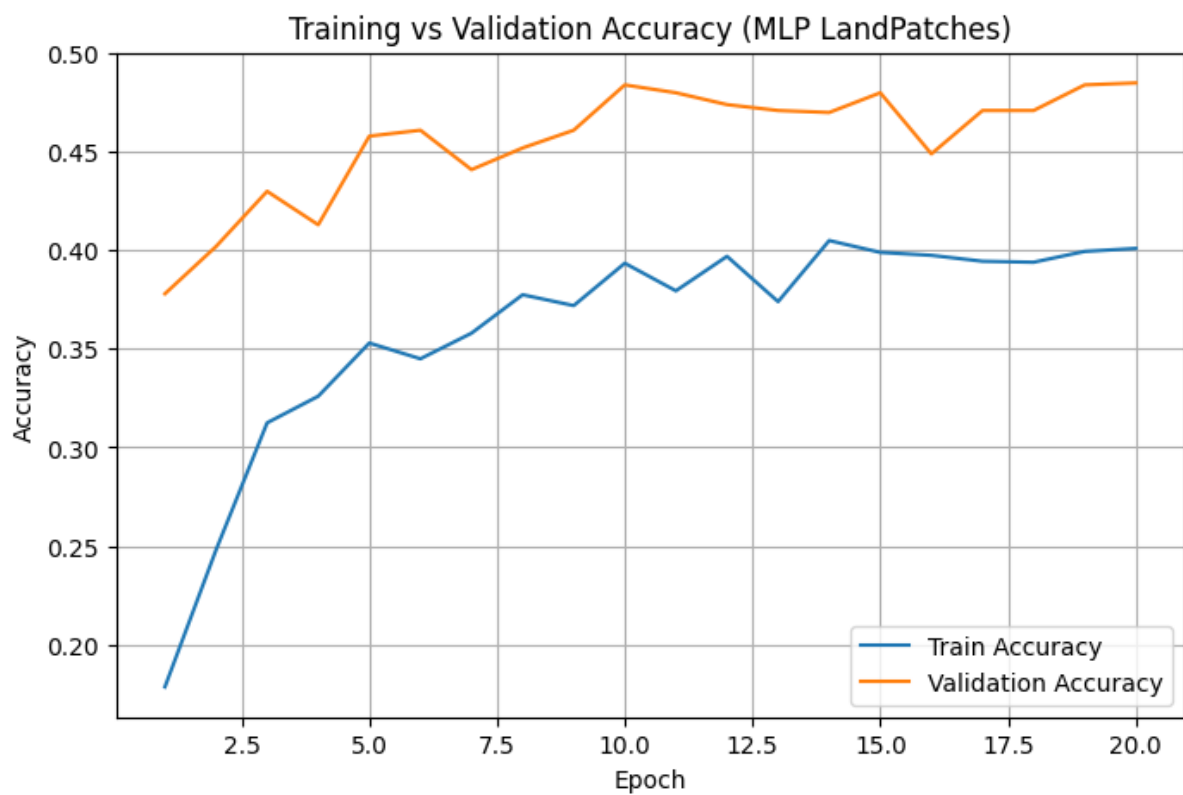
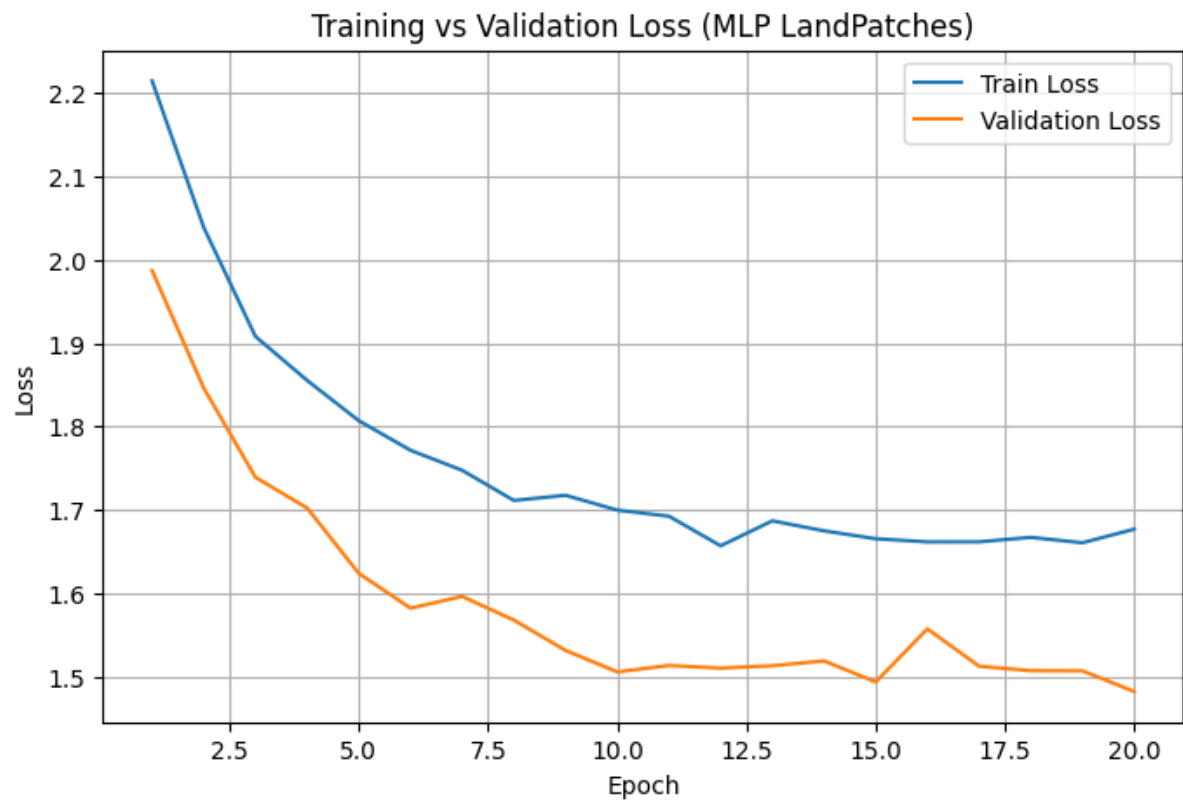


-With BatchNorm:

Epoch 20 | Train loss: 1.6770, acc: 0.4010 | Val loss: 1.4823, acc: 0.4850

Test Accuracy: 0.4747

Test F1-score (macro): 0.4508

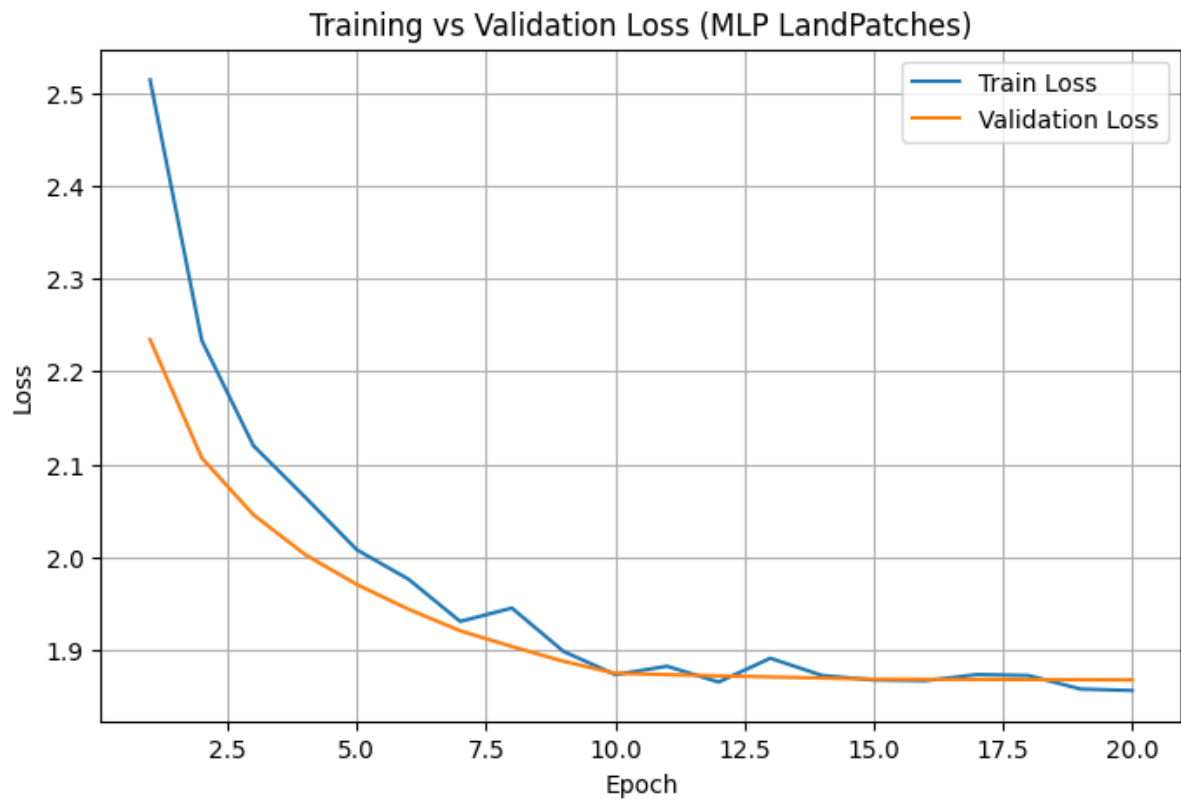


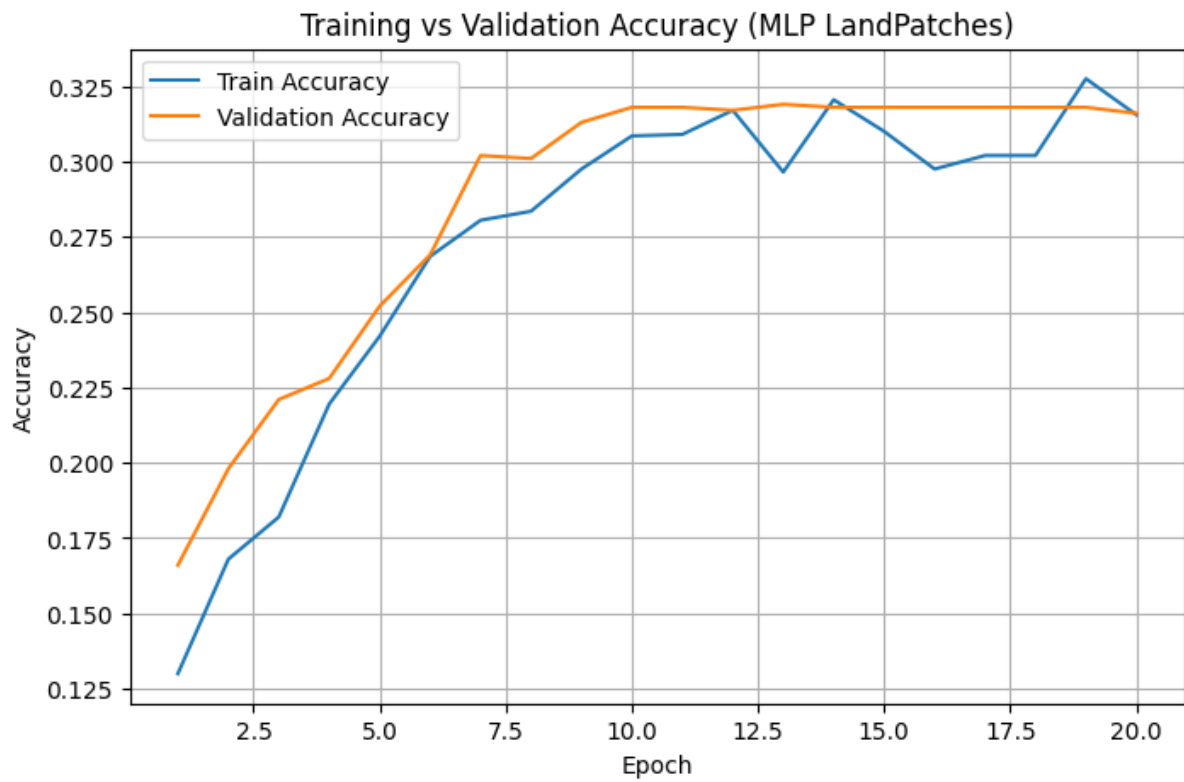
-With Augmentations:

Epoch 20 | Train loss: 1.8568, acc: 0.3155 | Val loss: 1.8683, acc: 0.3160

Test Accuracy: 0.3116

Test F1-score (macro): 0.2720



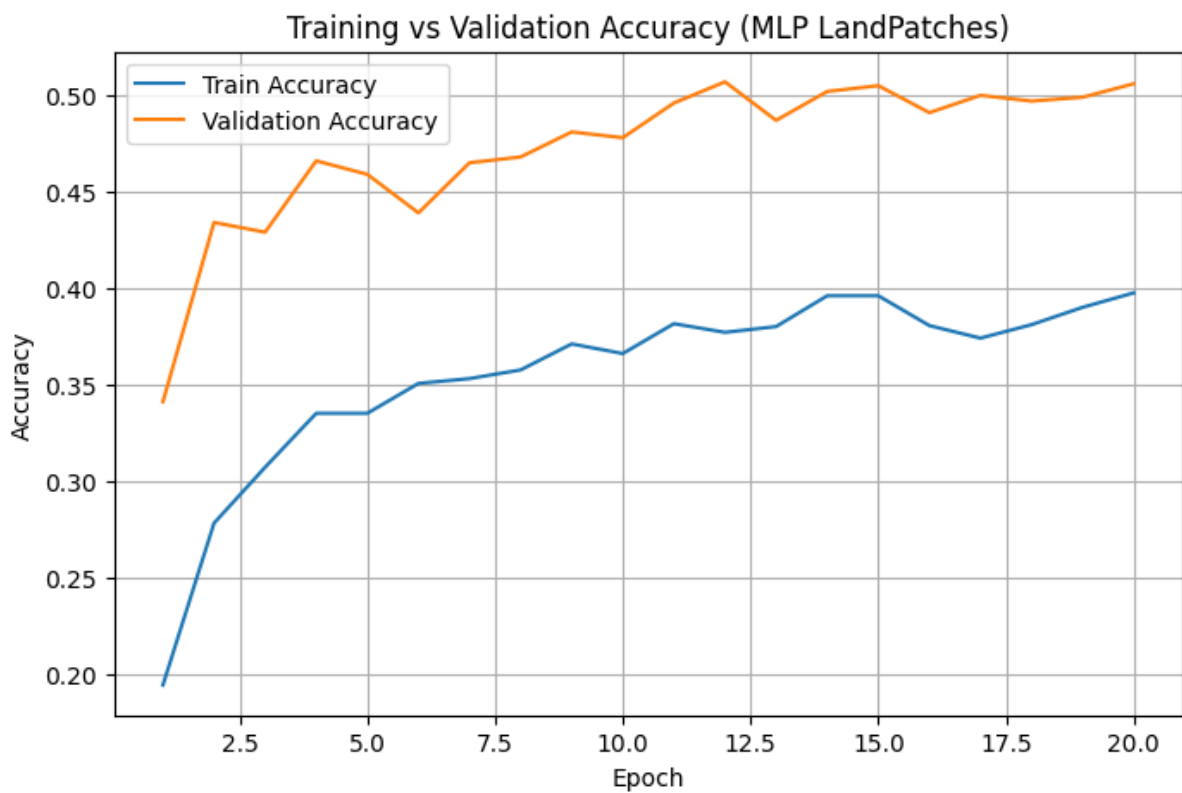
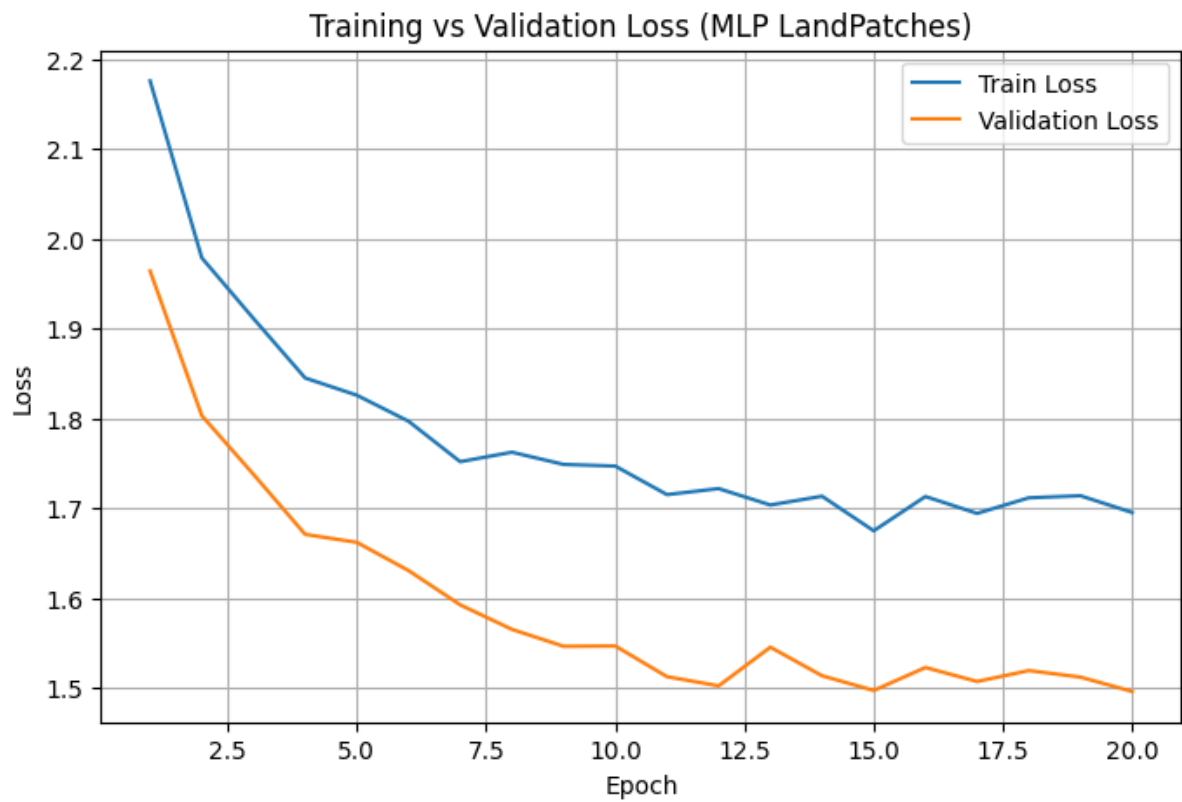


-With Both:

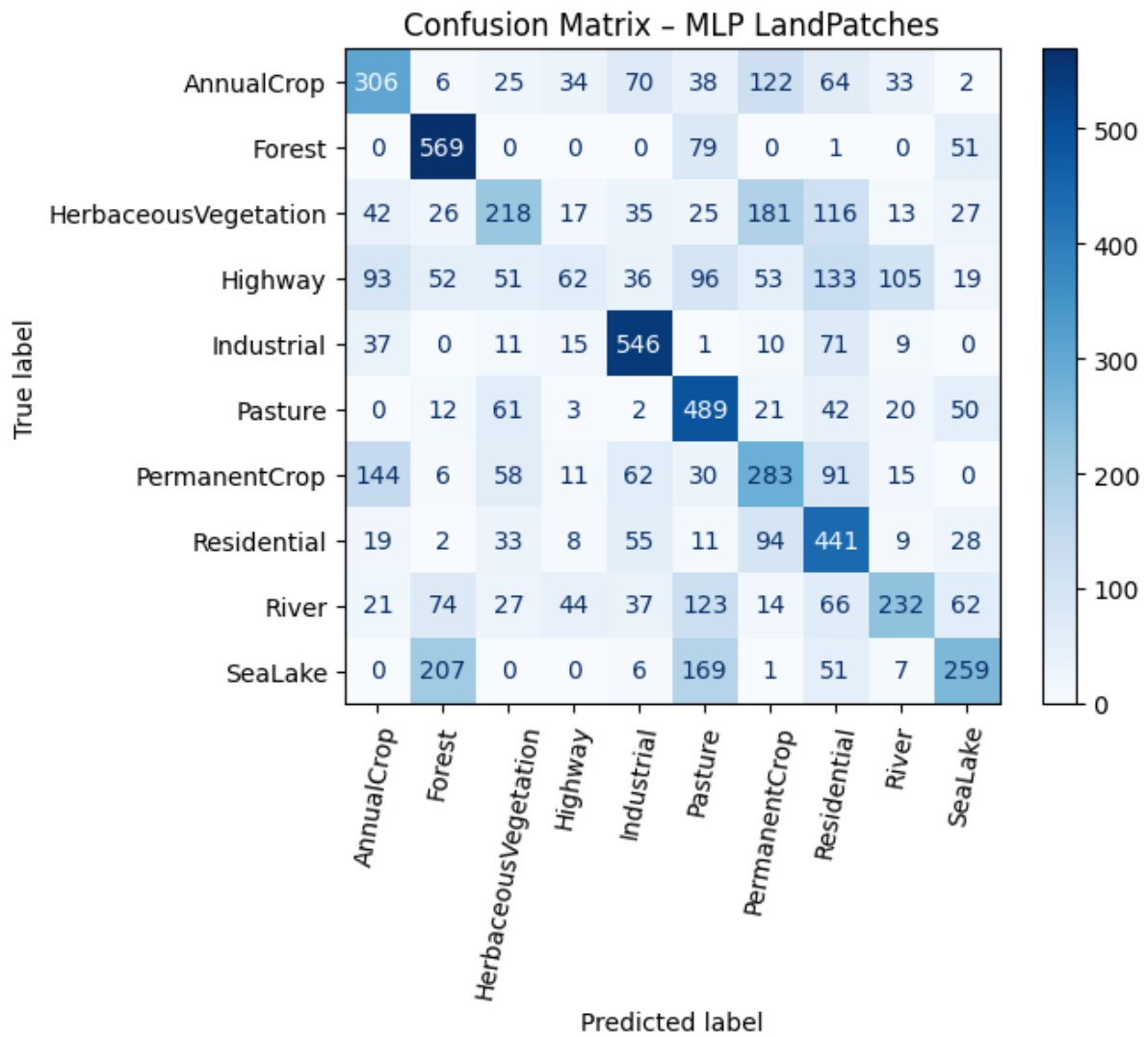
Epoch 20 | Train loss: 1.6950, acc: 0.3975 | Val loss: 1.4957, acc: 0.5060

Test Accuracy: 0.4864

Test F1-score (macro): 0.4622



Best: Both



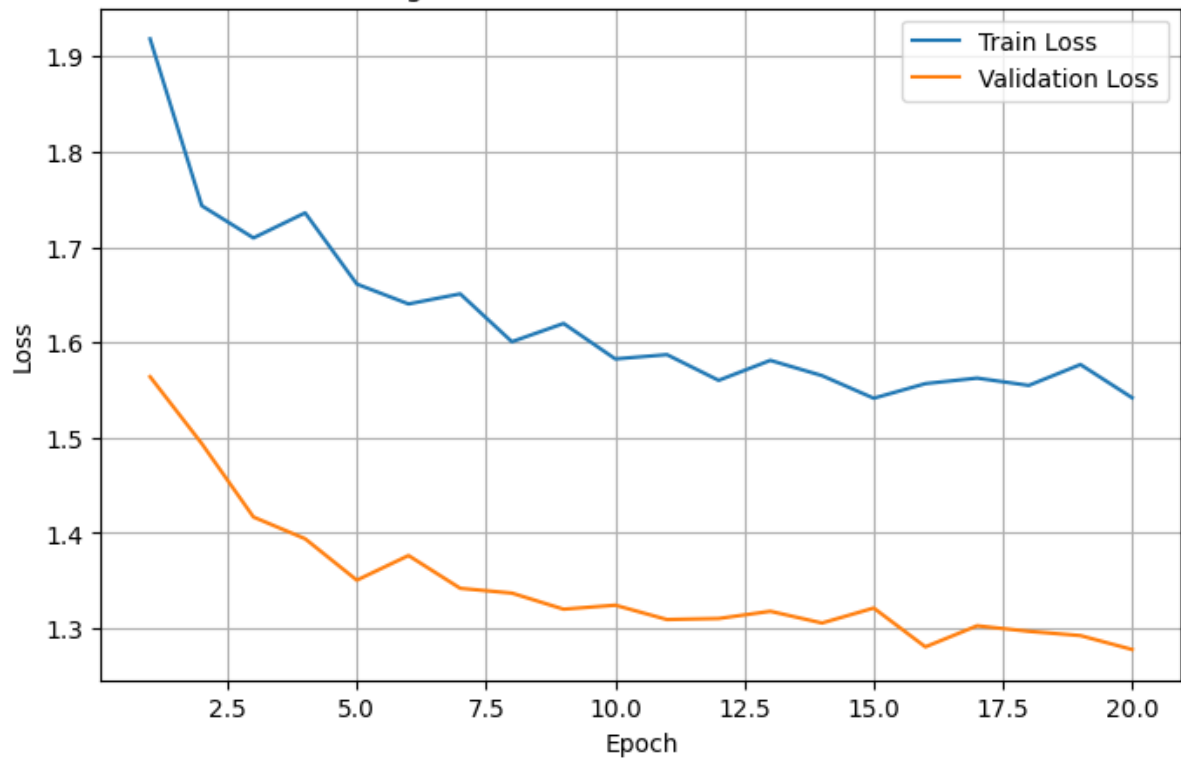
Improvement: Increased LR to 1e-3.

Epoch 20 | Train loss: 1.5419, acc: 0.4350 | Val loss: 1.2777, acc: 0.5670

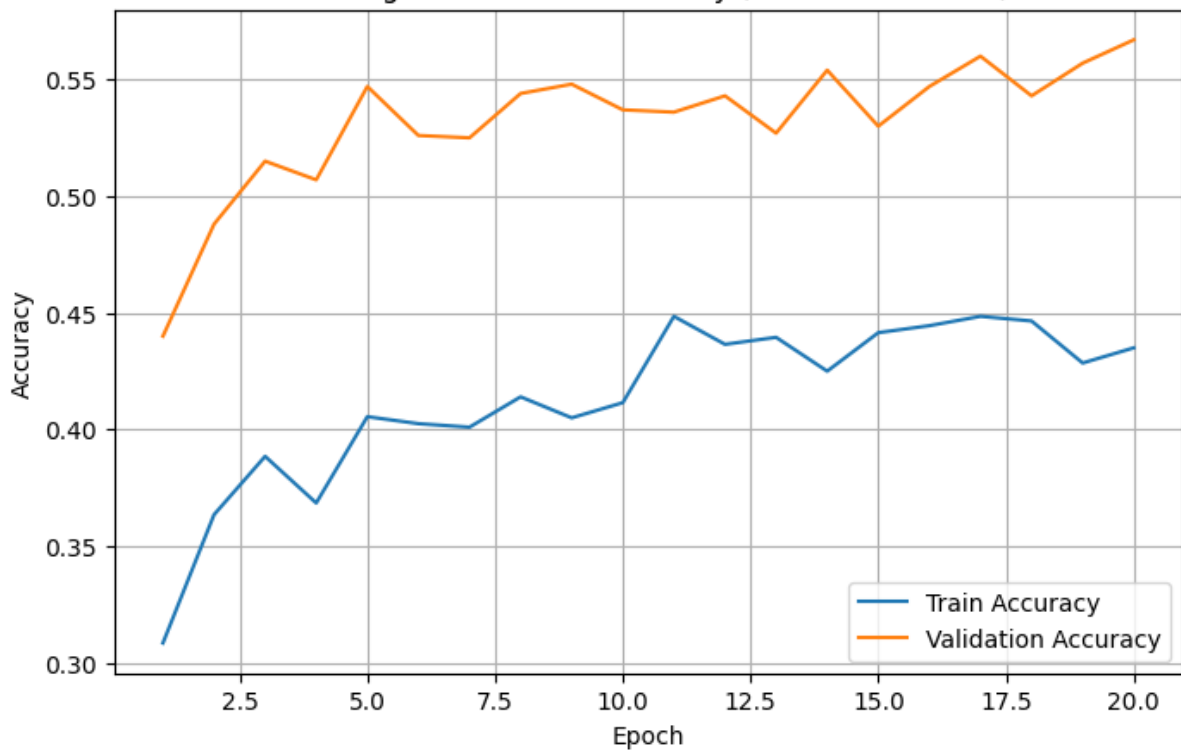
Test Accuracy: 0.5574

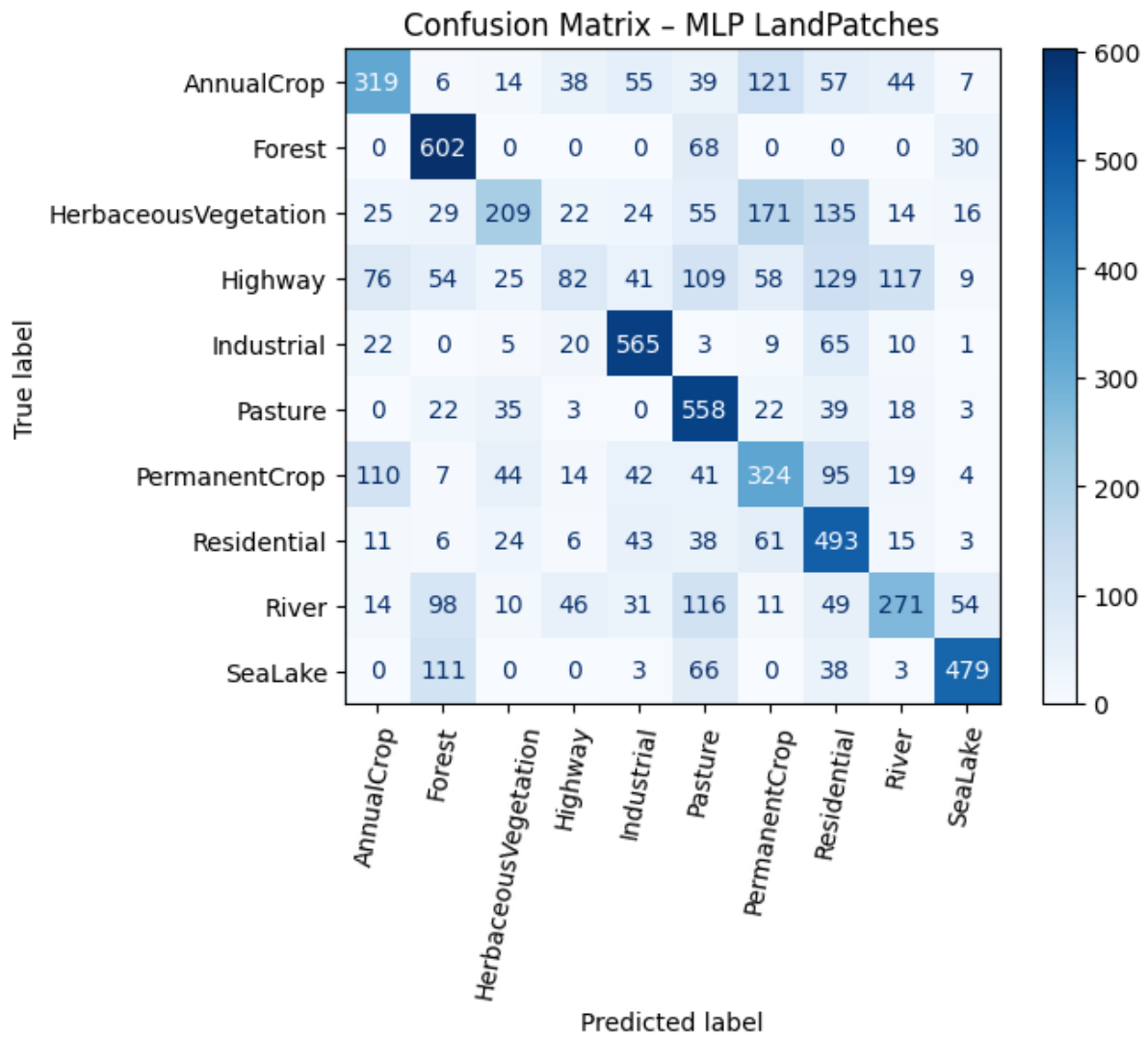
Test F1-score (macro): 0.5346

Training vs Validation Loss (MLP LandPatches)



Training vs Validation Accuracy (MLP LandPatches)





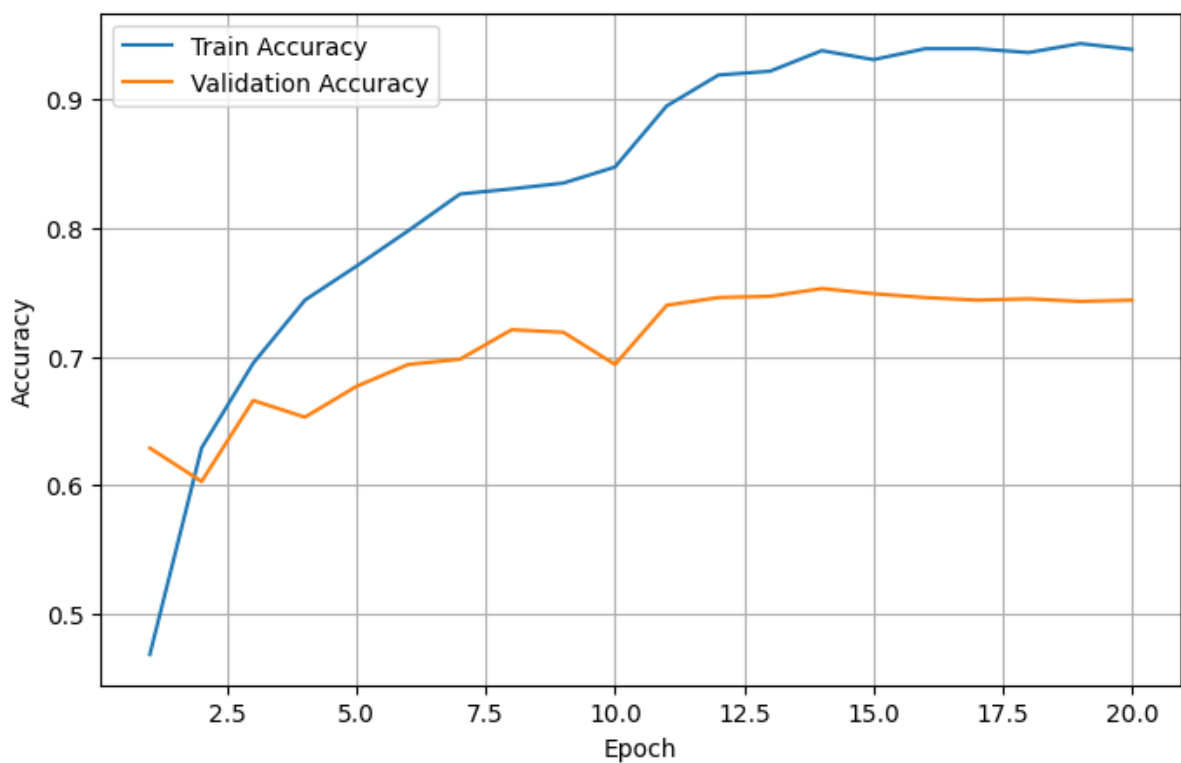
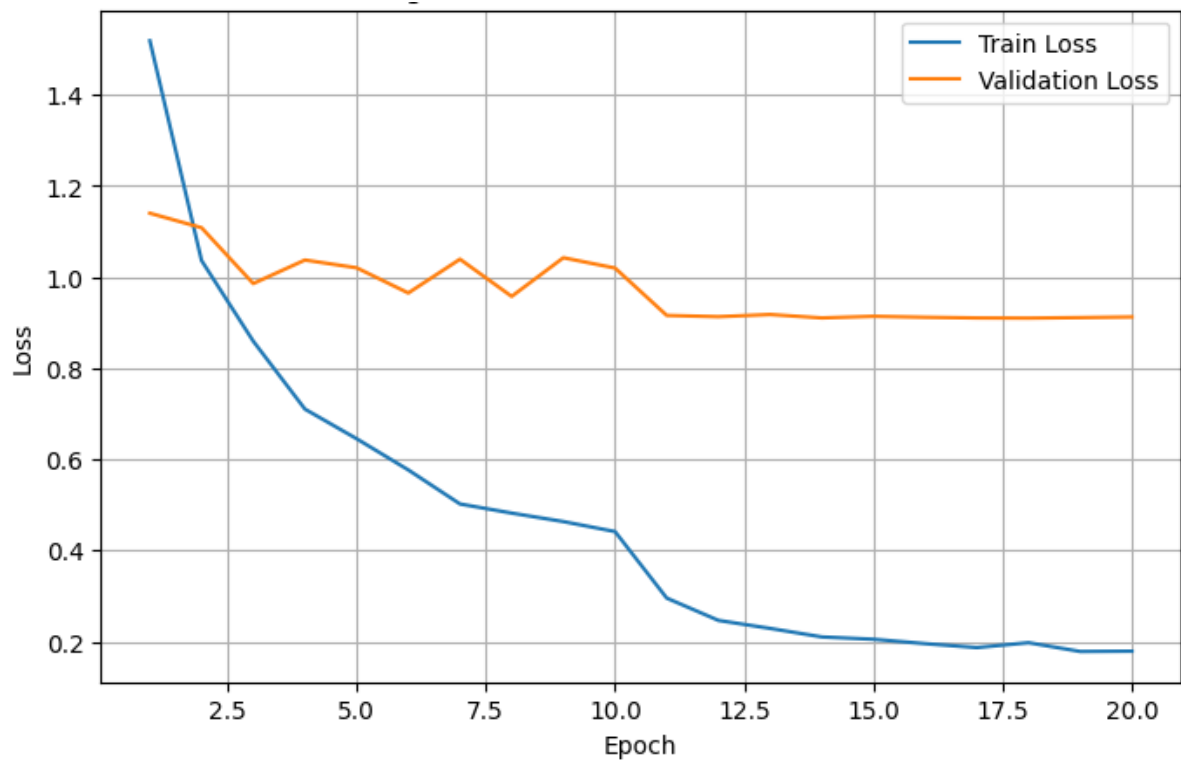
CNN_LandPatches

-Simple:

Epoch 20 | Train loss: 0.1789, acc: 0.9390 | Val loss: 0.9125, acc: 0.7440

Test Accuracy: 0.7340

Test F1-score (macro): 0.7323

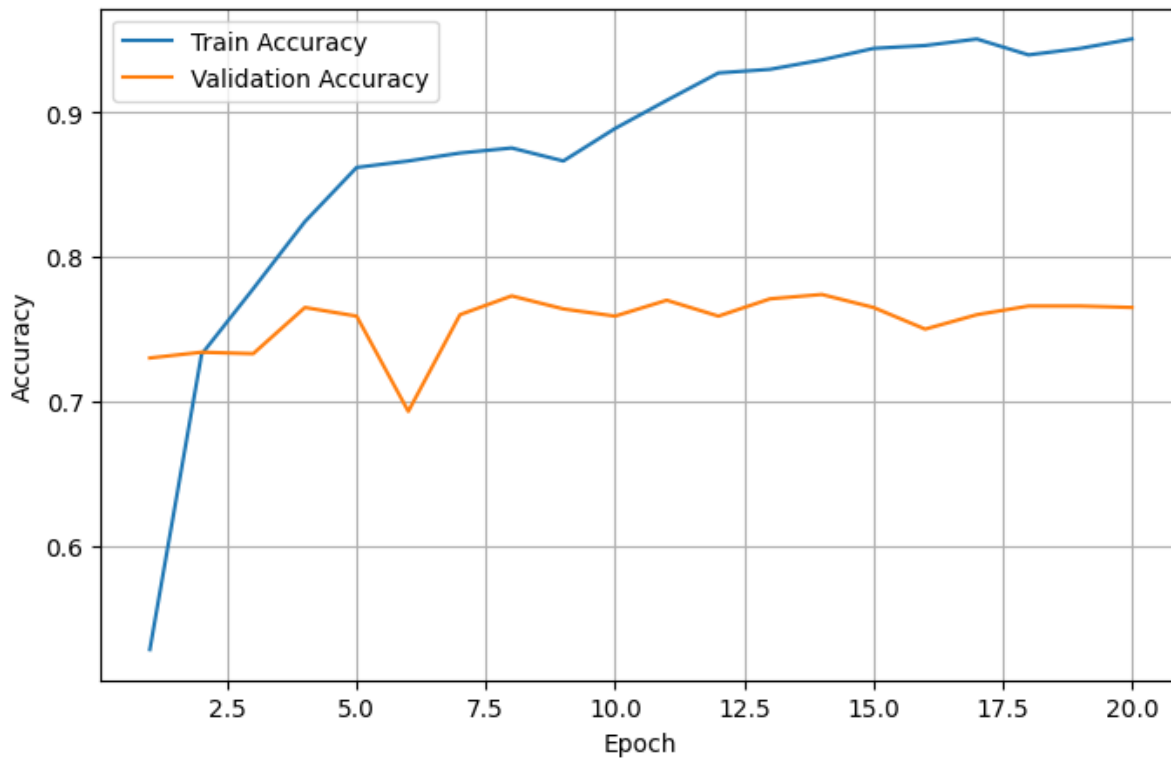
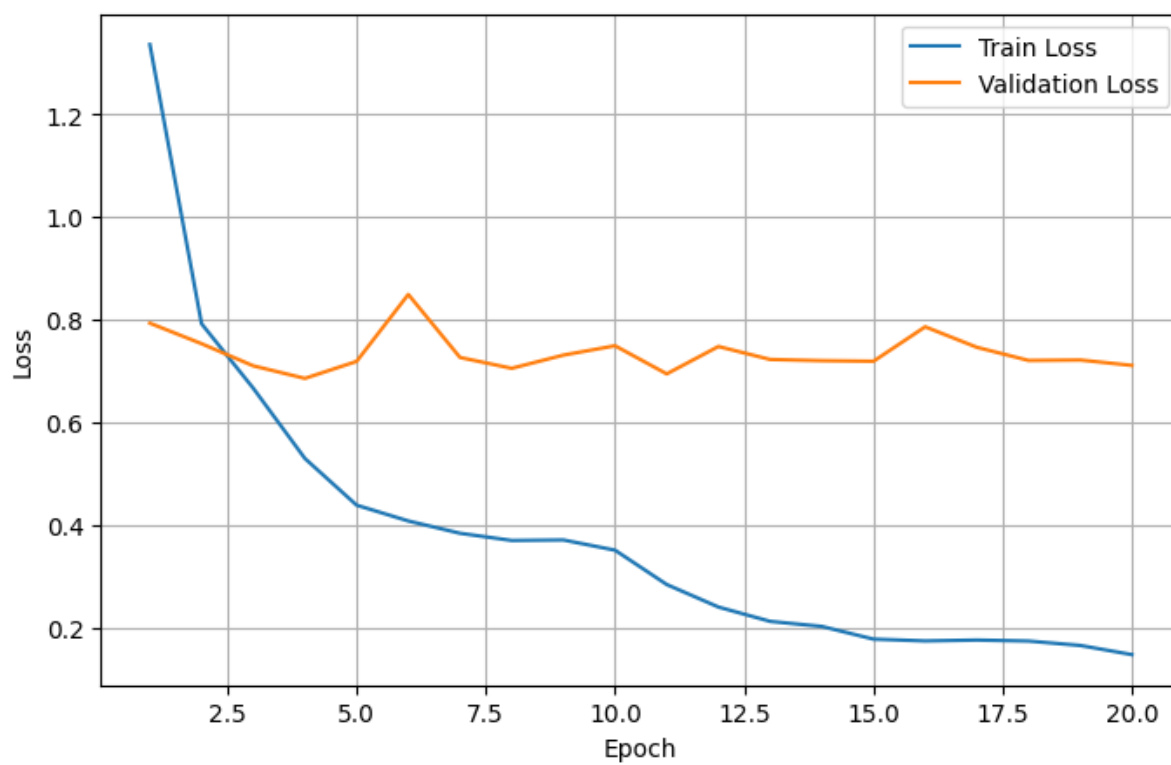


-With BatchNorm:

Epoch 20 | Train loss: 0.1490, acc: 0.9510 | Val loss: 0.7118, acc: 0.7650

Test Accuracy: 0.7690

Test F1-score (macro): 0.7637

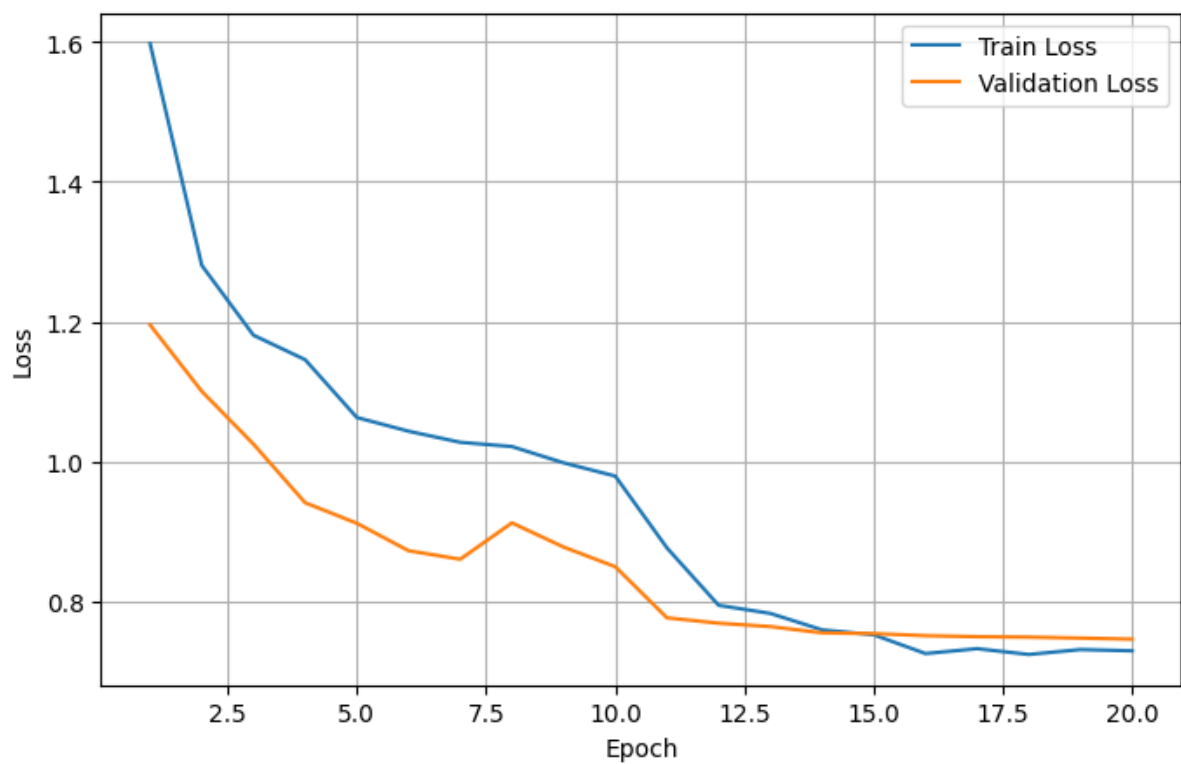


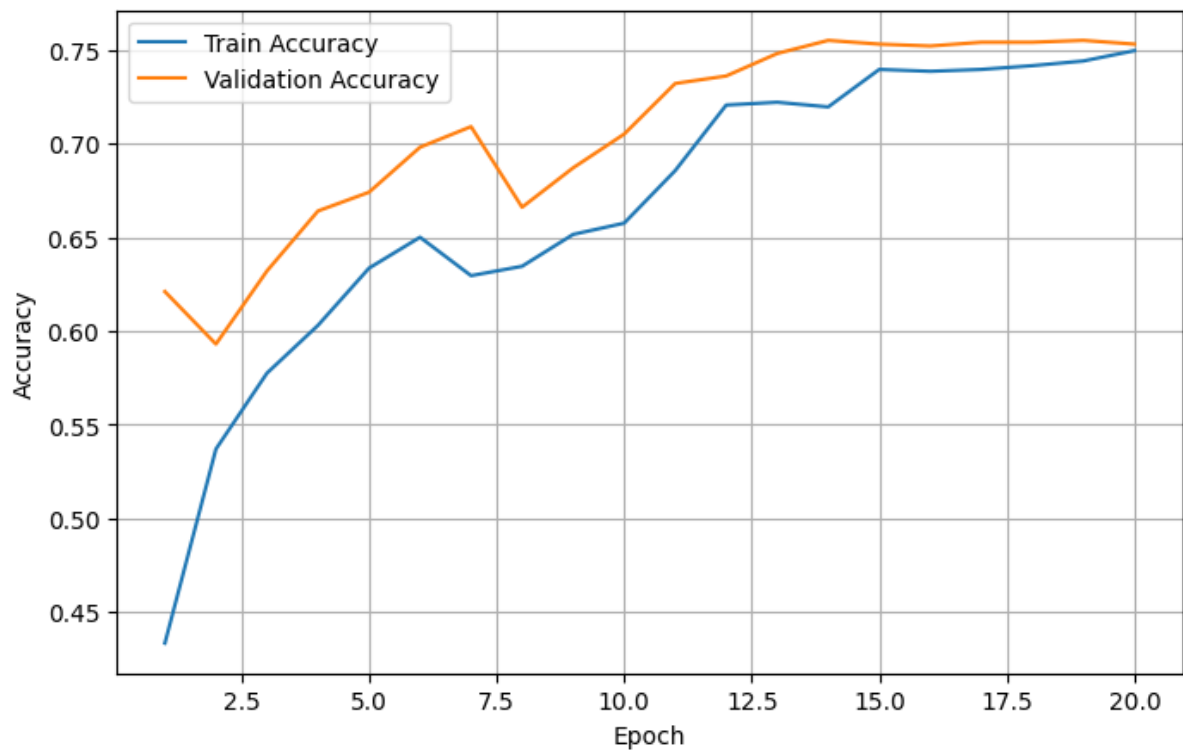
-With Augumentations:

Epoch 20 | Train loss: 0.7288, acc: 0.7495 | Val loss: 0.7454, acc: 0.7530

Test Accuracy: 0.7434

Test F1-score (macro): 0.7397



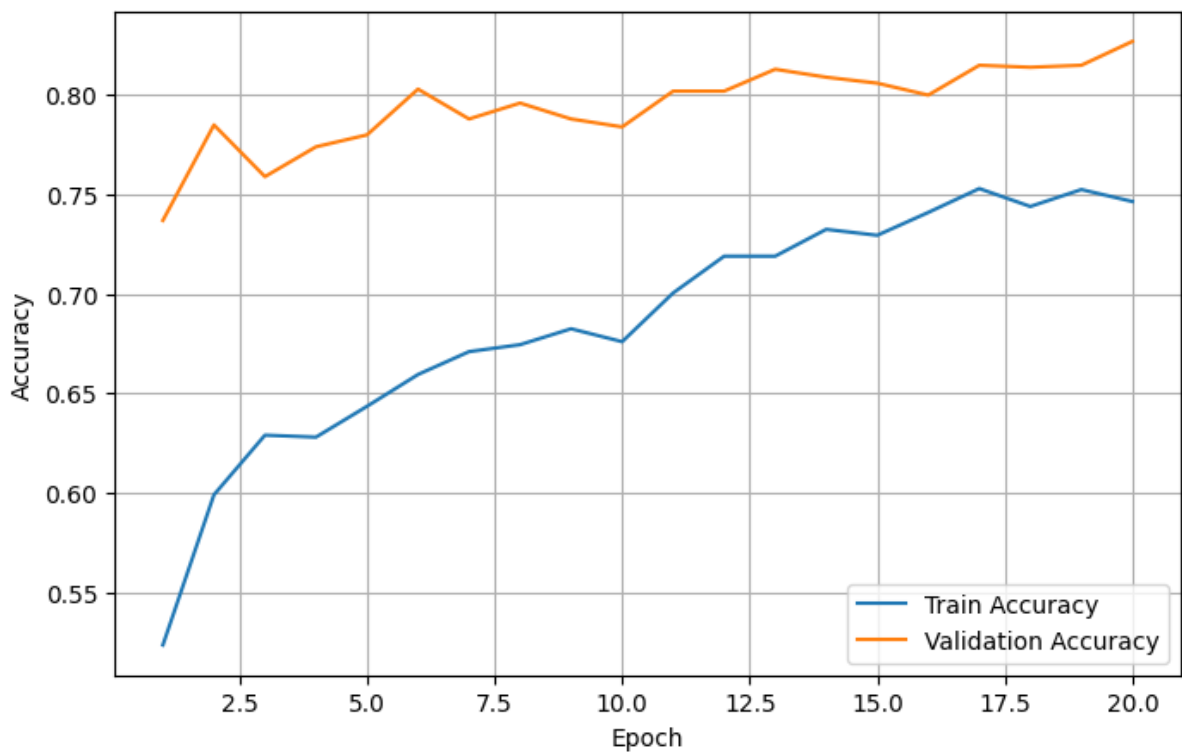
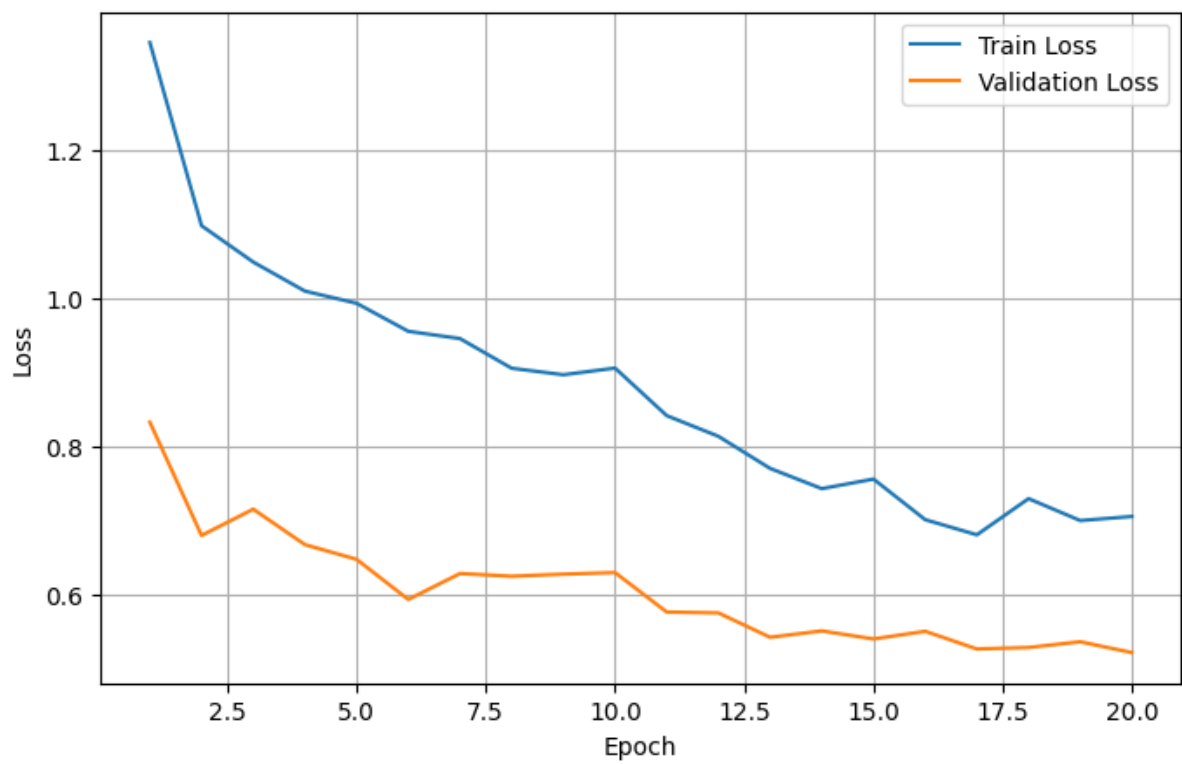


-With Both:

Epoch 20 | Train loss: 0.7051, acc: 0.7465 | Val loss: 0.5215, acc: 0.8270

Test Accuracy: 0.8129

Test F1-score (macro): 0.8122



Best: With Both:

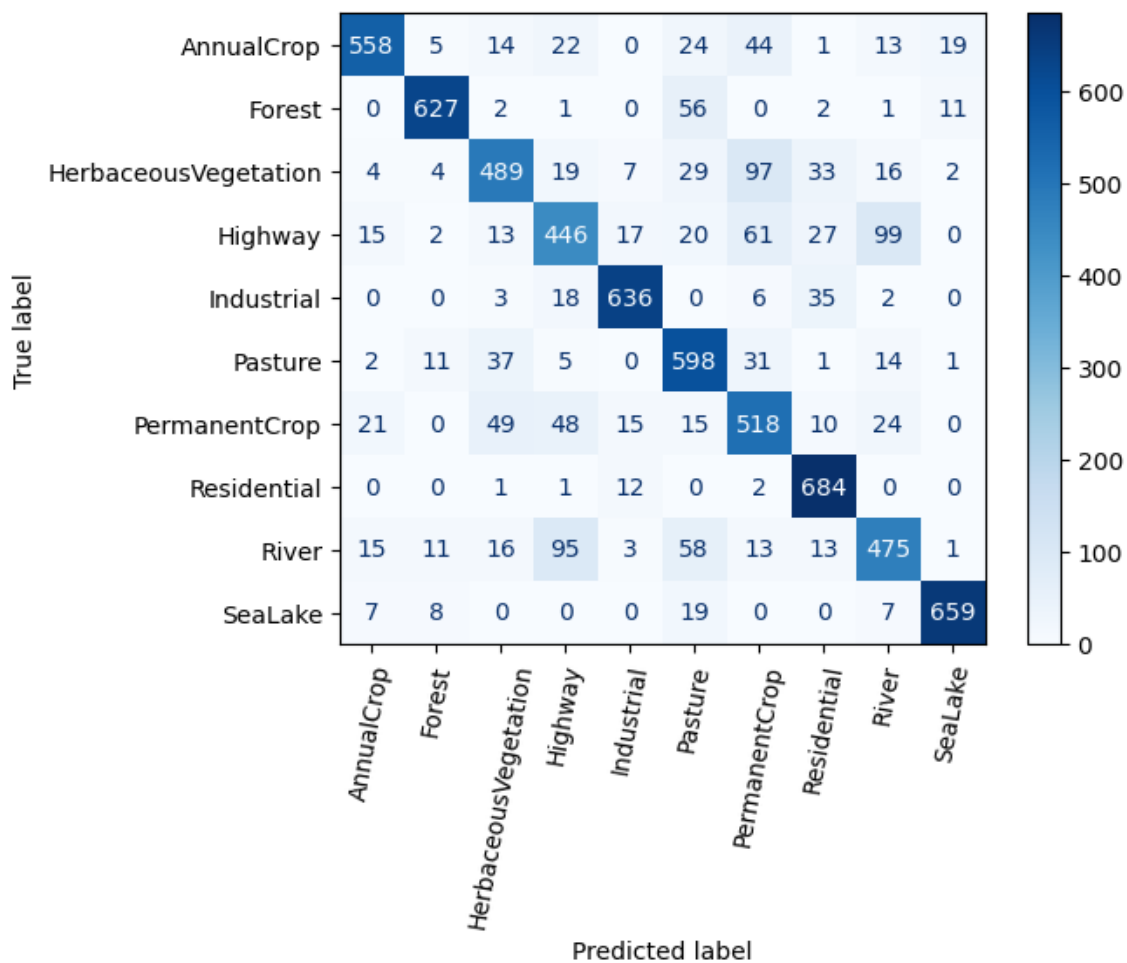


Table of land patches:

Configuration	Acuratete maxima train	Acuratete test
MLP Simple	0.3545	0.3834
MLP With augumentations	0.3155	0.3116
MLP With Batchnorm	0.4010	0.4747
MLP With both	0.3975	0.4864
MLP With both and higher lr	0.4350	0.5574
CNN Simple	0.9390	0.7340
CNN With augumentations	0.7495	0.7434
CNN With Batchnorm	0.9510	0.7690
CNN With both	0.7465	0.8129

CNN performed much better than MLP in transfer learning. MLP tends to underfit and performs better with higher learning rates. CNN does overfitting without augmentation, but it still gets good results.